

FlowerNet: Custom Flower Image Classification with Five CNN Models

A custom dataset of 300 flower images — daisy, rose and sunflower, 100 images per class — is classified with five deep learning models under 5-fold stratified cross-validation. The project applies data augmentation, compares five complementary evaluation metrics, and visualises learning curves to detect overfitting.

Course Assignment Report

Computer Vision & Deep Learning project — dataset collection, model implementation, cross-validated training and evaluation were performed end-to-end for this assignment.

300
IMAGES

3
CLASSES

5
MODELS

5-Fold
CROSS-VALIDATION

99.0%
BEST CV ACCURACY

Table of Contents

1. Introduction and Project Overview	2
2. Project Description	3
2.1 Dataset Description	3
2.2 Data Augmentation	4
2.3 The Five Models and Their Parameters	5
2.4 Why Cross-Validation?	6
2.5 Why Learning Curves?	6
2.6 Which Evaluation Metrics and Why?	7
2.7 Why These Five Models?	7
3. Implementation (Source Code)	8
4. Results and Discussion	43
4.1 Cross-Validated Comparison of the Five Models	43
4.2 Fold Stability	45
4.3 Learning Curves and Overfitting Analysis	46
4.4 Confusion Matrices and ROC Analysis	49
5. Conclusion	53
References	53

1. Introduction and Project Overview

1.1 Project Name

The name of this project is **FlowerNet — Custom Flower Image Classification with Five CNN Models**. FlowerNet is an end-to-end deep learning project that classifies flower photographs into three species (daisy, rose and sunflower) using a custom-built dataset, five different convolutional neural network architectures, 5-fold stratified cross-validation, data augmentation and a suite of complementary evaluation metrics. The complete experiment is automated by a single Python pipeline so that every number and every figure in this report can be reproduced with one command.

1.2 Objective and Requirements Coverage

The assignment asks for an image-classification project built on a custom dataset with a minimum of three classes, the application of at least three (here five) models, cross-validation, data augmentation, proper evaluation metrics for model comparison, and learning curves that visualise whether the models overfit. Table 1 maps every formal requirement to the concrete artefact of this project that satisfies it. All six requirements are fully covered.

#	Requirement	How it is satisfied in FlowerNet
1	Custom dataset (≥ 3 classes)	Self-collected custom dataset: 300 images, 3 classes $\times$ 100 images (daisy, rose, sunflower), deduplicated and normalised to 224 $\times$ 224 px
2	Cross-validation	5-fold stratified cross-validation (scikit-learn StratifiedKFold, seed 42); every image is used for validation exactly once
3	At least 3 models	Five models: SimpleCNN (from scratch) + VGG16, ResNet50, MobileNetV2, EfficientNet-B0 (ImageNet-pretrained, frozen backbone + trainable head)
4	Data augmentation	RandomResizedCrop, HorizontalFlip, Rotation $\pm 15^\circ$, ColorJitter — applied to training data only; augmented copies also pass through the frozen backbones
5	Proper evaluation metrics	Accuracy, macro Precision, macro Recall, macro F1-score, macro ROC-AUC, confusion matrices — reported per fold and aggregated (mean $\pm$ std)
6	Learning curves + overfitting visualisation	Epoch-wise train/val loss and accuracy curves per model; train–validation accuracy gap chart; accuracy-vs-training-set-size curve

Table 1. Assignment requirements and where they are satisfied.

2. Project Description

2.1 Dataset Description

FlowerNet uses a **custom dataset** that was collected specifically for this assignment rather than an off-the-shelf benchmark. It contains **300 RGB images of flowers** divided into **three classes** — **daisy, rose and sunflower** — **with exactly 100 images per class**, so the dataset is perfectly balanced. The raw photographs were gathered from web image search with multiple, deliberately varied text queries per class (close-ups, gardens, bouquets, fields, different colours and backgrounds) so that each class covers substantial visual variety instead of one stereotypical view. Exact duplicate images were removed by MD5 content hashing, and every image was verified with PIL before being accepted.

All accepted images were resized so that the short side is 256 pixels and then centre-cropped to **224 × 224 pixels**, the canonical input resolution of the ImageNet-pretrained backbones used in this project. A uniform resolution removes one source of noise, keeps the memory footprint predictable, and means that the only geometric variation left in the data is real-world variation — which the augmentation pipeline of Section 2.2 then explicitly models. Figure 2 shows the class distribution and a random sample of images from each class.

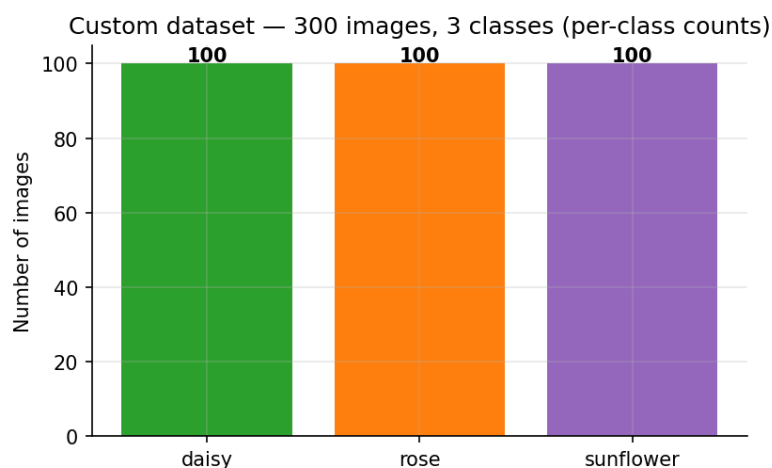


Figure 1. Class distribution of the custom dataset — 3 classes with exactly 100 images each (300 total).

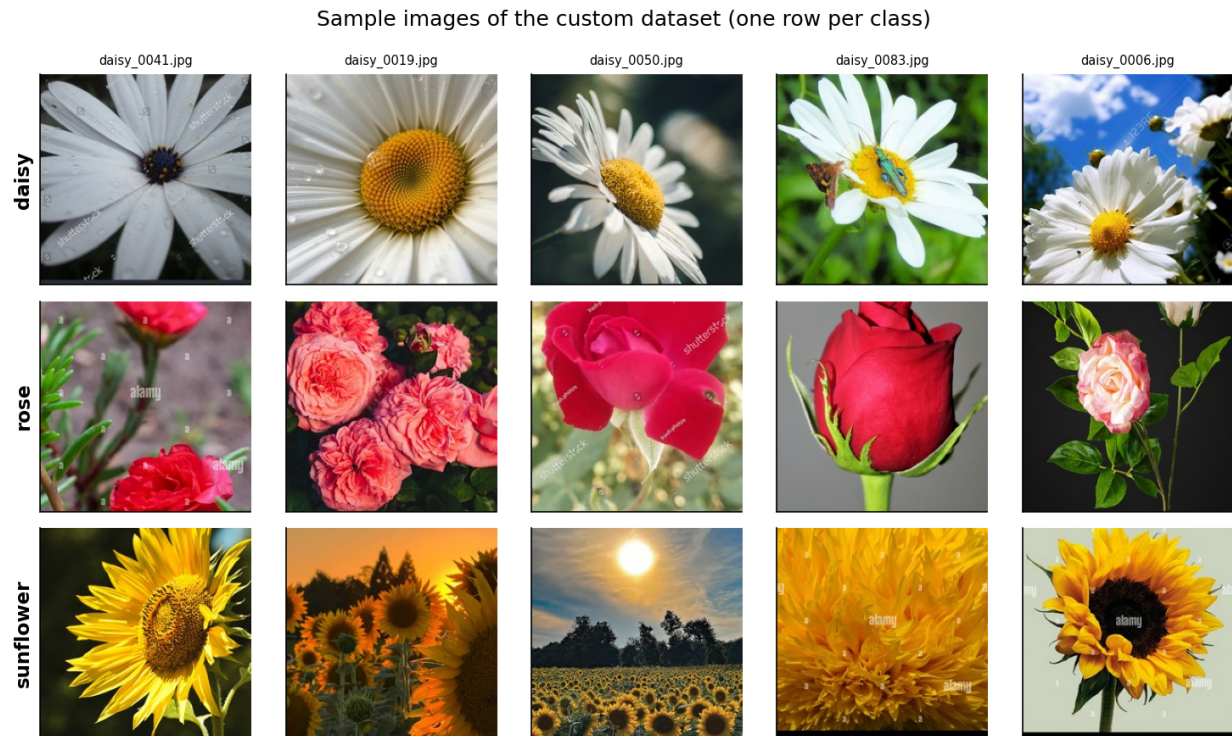


Figure 2. Randomly chosen sample images, one row per class. Note the large variation in background, pose, colour and scale within every class.

Class	Images	Resolution	Typical visual characteristics
daisy	100	224 × 224	white radial petals around a bright yellow centre, thin stems
rose	100	224 × 224	layered, spiral petals; red / pink / yellow colour variants
sunflower	100	224 × 224	large yellow head, dark central disc, coarse leaves

Table 2. Dataset summary (100 images per class, 300 images in total).

2.2 Data Augmentation

Because 300 images is a small dataset by deep-learning standards, FlowerNet applies **data augmentation** to enlarge the effective training set without collecting new photographs. The augmentation pipeline (applied to **training data only**, never to validation data) combines four random operations: **RandomResizedCrop** (crop a random region covering 75–100 % of the image, then resize back to the target resolution), **RandomHorizontalFlip** ($p = 0.5$), **RandomRotation** ($\pm 15^\circ$) and **ColorJitter** (brightness, contrast and saturation each jittered by up to 25 %). These operations simulate the exact variations that a real camera produces (different viewpoints, distances, orientations and lighting conditions) and therefore teach the models to be invariant to them.

For the four transfer-learning models the frozen backbones convert images into fixed 1280–2048-dimensional feature vectors. Augmentation is injected by passing every training image through the backbone **three times**: once with the deterministic evaluation transform (the clean view) and twice with independent random augmentation passes. Each classifier head is therefore trained on the union of clean and

augmented feature vectors, which multiplies the effective training set by three and acts as a strong regulariser. The from-scratch SimpleCNN instead receives freshly augmented pixel images on every epoch. Figure 3 visualises typical augmented views: random crops and rotations change the viewpoint, colour jitter simulates different cameras and lighting.



Figure 3. The augmentation pipeline in action: each row shows one original image and three random augmented views (random crop, flip, rotation and colour jitter).

2.3 The Five Models and Their Parameters

FlowerNet compares **five models** that deliberately span the design space of modern image classification. **SimpleCNN** is a compact convolutional network designed and trained **from scratch**: three Conv3×3→BatchNorm→ReLU→MaxPool blocks with 32, 64 and 128 filters, followed by global average pooling and a 128-unit fully-connected head. It has only ≈0.11 M parameters and serves as the baseline that shows how far a small network gets on 300 images without any prior knowledge.

The remaining four models apply **transfer learning**: their convolutional backbones are **ImageNet-pretrained and frozen**, so they act as fixed feature extractors that convert each 224×224 image into a rich feature vector; only a small trainable head (Dropout → Linear → ReLU → Dropout → Linear, 128 hidden units) is fitted on those features under cross-validation. This two-stage strategy is CPU-friendly and avoids overfitting the tiny dataset, because 94–99 % of each network's parameters are never updated. The four backbones represent four milestones of CNN design: **VGG16** (2014, uniform 3×3 convolution stacks), **ResNet50** (2016, bottleneck residual blocks with skip connections), **MobileNetV2** (2018, depthwise-separable inverted residuals for mobile devices) and **EfficientNet-B0** (2019, compound-scaled mobile inverted bottlenecks).

Model	Type	Params (total)	Params (trainable)	Input	Architecture highlights
SimpleCNN	scratch	0.11 M	0.11 M (100 %)	128×128	3 conv blocks (32-64-128) + GAP + FC-128 head, trained from scratch
VGG16	transfer	14.78 M	66 K (0.4 %)	224×224	13 conv layers of uniform 3×3 filters, ImageNet-pretrained, frozen
ResNet50	transfer	23.77 M	263 K (1.1 %)	224×224	bottleneck residual blocks with identity skip connections, frozen
MobileNetV2	transfer	2.39 M	164 K (6.9 %)	224×224	depthwise-separable inverted residual blocks, frozen
EfficientNet-B0	transfer	4.17 M	164 K (3.9 %)	224×224	compound-scaled MBConv blocks, frozen

Table 3. The five models and their parameter budgets. Transfer heads are the small MLPs trained on frozen backbone features.

All models share the same training protocol: Adam optimiser with learning rate 10^{-3} , cross-entropy loss, batch size 64 for the transfer heads (30 epochs) and batch size 32 for SimpleCNN (12 epochs at 128×128 input). The best epoch is selected by validation accuracy within every cross-validation fold, and the metrics of the best epoch are reported.

2.4 Why Cross-Validation?

With only 300 images, a single train/validation split would be fragile: 20 % of the data is just 60 images, so one lucky or unlucky split could change the measured accuracy by several percentage points and make the five models impossible to rank fairly. **K-fold cross-validation** solves this by splitting the dataset into $K = 5$ stratified folds — stratified meaning every fold keeps the exact 1 : 1 : 1 class ratio — and training K separate models, each time holding out a different fold for validation. Every image plays the role of a validation image exactly once, so the final out-of-fold prediction covers the whole dataset, and the reported metric is the **mean $\pm$ standard deviation over five folds**. The standard deviation doubles as an estimate of how sensitive each model is to the training subset, which is exactly the kind of stability information a small dataset demands. Cross-validation also uses the limited data far more efficiently than a single split (each model trains on 80 % of the data five times instead of once), and it removes the need to sacrifice a separate permanent test set, which would be prohibitively expensive at this dataset size.

2.5 Why Learning Curves?

A single accuracy number cannot tell us *how* a model behaves during training, so FlowerNet visualises two complementary kinds of **learning curves**. First, **epoch-wise curves** plot training loss/accuracy and validation loss/accuracy against training epochs, averaged over the five folds. The vertical distance between the training and validation curves is the classic diagnostic of **overfitting**: a model that memorises its training data shows training accuracy climbing towards 100 % while validation accuracy stalls or falls and validation loss starts to rise again. Underfitting, conversely, shows both curves stuck at a low level. Second, a **training-set-size learning curve** plots validation accuracy against the number of training images used (25 %, 50 %, 75 %, 100 %). This curve answers two questions at once: whether the models have already saturated

the information content of the dataset (curve flat $\rightarrow$ more data would not help much) and whether collecting more images would be worthwhile (curve still climbing $\rightarrow$ more data would help). Plotting the train–validation gap against training-set size additionally shows how data volume suppresses overfitting, which is precisely the visualisation the assignment asks for.

2.6 Which Evaluation Metrics and Why?

No single metric tells the whole story of a classifier, so FlowerNet reports five complementary metrics — always computed on the held-out fold, never on training data — and aggregates them over the five cross-validation folds:

- **Accuracy** — the overall fraction of correctly classified images; intuitive and, because the dataset is perfectly balanced (100/100/100), not misleading here.
- **Precision (macro)** — of the images predicted as class c , how many really belong to c , averaged over classes; penalises false alarms for each flower.
- **Recall (macro)** — of the images of class c , how many were found; penalises missed flowers. A model can trade precision against recall, so both are needed.
- **F1-score (macro)** — the harmonic mean of precision and recall; our primary model-ranking metric because it summarises both error types in one number and treats all three classes equally regardless of size.
- **ROC-AUC (macro, one-vs-rest)** — the probability that a random positive image of a class receives a higher predicted score than a random negative; evaluates the *ranking quality* of the predicted probabilities, not just the hard arg-max decision, and is threshold-independent.

Finally, **confusion matrices** (row-normalised, computed from the pooled out-of-fold predictions) reveal *which* classes are mixed up with each other, information that scalar metrics hide. Macro averaging is used throughout so that every class contributes equally; with the balanced dataset this also coincides with micro averaging, making the numbers easy to interpret.

2.7 Why These Five Models?

The five models were chosen to answer the assignment's comparison question from as many angles as possible. SimpleCNN provides the **from-scratch baseline**: it shows what is achievable with ≈ 0.1 M parameters and no external knowledge, and its training dynamics illustrate overfitting behaviour on a small dataset. The four pretrained backbones then isolate the value of **transfer learning**. VGG16 is the historical reference architecture whose uniform structure makes it the textbook choice for feature extraction. ResNet50 adds residual learning and is one of the most widely used general-purpose backbones in computer vision. MobileNetV2 and EfficientNet-B0 represent the modern **light-weight** family: if a 2–4 M-parameter mobile model matches a 24 M-parameter classic, that is a practically relevant finding for deployment. Comparing all five under the identical cross-validation, augmentation and metrics protocol therefore measures not only accuracy differences but also the accuracy-per-parameter trade-off.

3. Implementation (Source Code)

The project is implemented in **Python 3.12** with **PyTorch / torchvision** for models and training, **scikit-learn** for cross-validation and metrics, **NumPy / PIL** for data handling and **matplotlib** for all figures. The pipeline is fully scripted and reproducible: *main.py* runs dataset verification, feature extraction, cross-validated training of all five models and figure generation in order. The following pages show screenshots of the complete source code of every module, exactly as executed for this report.

File	Lines	Role in the pipeline
config.py	58	central configuration: paths, image sizes, CV folds, hyper-parameters
data.py	121	dataset scanning, eval + augmentation transforms, PyTorch datasets
models.py	124	SimpleCNN definition, frozen backbones, classifier heads
extract_features.py	95	stage 1 — frozen-backbone feature extraction (clean + augmented)
train_cv.py	343	stage 2 — 5-fold CV training loop, learning-curve data collection
evaluate.py	383	stage 3 — metrics, confusion matrices, ROC curves, all figures
main.py	66	one-command experiment driver (dataset → features → CV → figures)
prepare_dataset.py	62	raw-image cleaning: resize + centre-crop to 224×224
scripts/collect_dataset.py	163	custom dataset collection: search, download, validate, deduplicate

Table 4. Project structure — every file is shown as a screenshot below.

Listing 1 — config.py (part 1/2). Central configuration: paths, image sizes, 5-fold CV setup, classifier-head and SimpleCNN hyper-parameters.

```

1  """
2  config.py – Central configuration for the FlowerNet project.
3
4  All hyper-parameters, paths and constants live here so that every other
5  module stays clean and the whole experiment is reproducible from one file.
6  """
7  import os
8
9  # ----- paths -----
10 PROJECT_DIR = os.path.dirname(os.path.abspath(__file__))
11 RAW_DATA_DIR = os.path.join(PROJECT_DIR, "dataset_raw")      # downloaded raw images
12 DATA_DIR = os.path.join(PROJECT_DIR, "dataset")            # final 224x224 images
13 RESULTS_DIR = os.path.join(PROJECT_DIR, "results")
14 FIGURES_DIR = os.path.join(PROJECT_DIR, "figures")
15 FEATURES_DIR = os.path.join(REULTS_DIR, "features")
16 LOGS_DIR = os.path.join(PROJECT_DIR, "logs")
17
18 # ----- data/image -----
19 IMG_SIZE = 224          # input resolution for ImageNet-pretrained backbones
20 CNN_INPUT_SIZE = 128    # input resolution for the from-scratch SimpleCNN
21 NUM_AUG_COPIES = 2      # augmented feature copies per image (transfer models)
22 MIN_DIM = 160           # discard downloaded images smaller than this
23
24 # ----- cross-validation setup -----
25 N_SPLITS = 5            # 5-fold stratified cross-validation
26 SEED = 42               # global random seed (reproducibility)
27
28 # ----- frozen-backbone extraction -----
29 BATCH_EXTRACT = 12      # batch size for backbone feature extraction (RAM-safe)
30
31 # ----- classifier-head hyperparams -----
32 HEAD_EPOCHS = 30        # epochs for the small MLP heads on frozen features
33 HEAD_LR = 1e-3
34 HEAD_BATCH = 64
35 HEAD_HIDDEN = 128
36
37 # ----- SimpleCNN hyperparams -----
38 CNN_EPOCHS = 12
39 CNN_BATCH = 32
40 CNN_LR = 1e-3
41
42 # ----- learning-curve settings -----

```

Listing 1 — config.py (part 2/2). Central configuration: paths, image sizes, 5-fold CV setup, classifier-head and SimpleCNN hyper-parameters.

```
43 SIZE_FRACTIONS = [0.25, 0.50, 0.75, 1.00] # training-set-size learning curve
44
45 # ----- ImageNet statistics -----
46 IMAGENET_MEAN = [0.485, 0.456, 0.406]
47 IMAGENET_STD = [0.229, 0.224, 0.225]
48
49 # ----- figure aesthetics -----
50 MODEL_COLORS = {
51     "SimpleCNN":      "#7f7f7f", # grey - from-scratch baseline
52     "VGG16":          "#d62728", # red - classic 2014 architecture
53     "ResNet50":       "#2ca02c", # green - residual connections
54     "MobileNetV2":    "#ff7f0e", # orange - lightweight mobile design
55     "EfficientNet-B0": "#9467bd", # purple - compound-scaled design
56 }
57
58 MODEL_ORDER = ["SimpleCNN", "VGG16", "ResNet50", "MobileNetV2", "EfficientNet-B0"]
```

Listing 2 — data.py (part 1/3). Dataset scanning, deterministic eval transform, data-augmentation transform, PyTorch Dataset classes.

```
1  """
2  data.py — Dataset loading, transforms (including augmentation) and helpers.
3
4  The dataset is a *custom* flower dataset collected for this project:
5      dataset/<class_name>/<class_name>_XXXX.jpg  (224x224 RGB JPEG)
6
7  Two transform pipelines are provided:
8      * eval transform    — deterministic resize + center crop (used for validation
9                          and for frozen-backbone feature extraction)
10     * augment transform — random resized crop / flip / rotation / color jitter
11                          (data augmentation, applied to TRAINING data only)
12  """
13  import os
14  import random
15
16  import numpy as np
17  import torch
18  from PIL import Image
19  from torch.utils.data import Dataset
20  from torchvision import transforms
21
22  from config import (DATA_DIR, IMG_SIZE, CNN_INPUT_SIZE, IMAGENET_MEAN,
23                     IMAGENET_STD, SEED, MIN_DIM)
24
25  # ----- seeding -----
26  def set_seed(seed: int = SEED):
27      """Seed python / numpy / torch for reproducibility."""
28      random.seed(seed)
29      np.random.seed(seed)
30      torch.manual_seed(seed)
31      torch.cuda.manual_seed_all(seed)
32
33  # ----- dataset listing -----
34  def list_images(data_dir: str = DATA_DIR):
35      """
36      Scan the dataset folder and return (paths, labels, class_names).
37      `class_names` is sorted alphabetically; label = index into class_names.
38      """
39      classes = sorted(d for d in os.listdir(data_dir)
40                      if os.path.isdir(os.path.join(data_dir, d)))
```

Listing 2 — data.py (part 2/3). Dataset scanning, deterministic eval transform, data-augmentation transform, PyTorch Dataset classes.

```

43     paths, labels = [], []
44     for label, cls in enumerate(classes):
45         cdir = os.path.join(data_dir, cls)
46         files = sorted(f for f in os.listdir(cdir) if f.lower().endswith(".jpg"))
47         for f in files:
48             paths.append(os.path.join(cdir, f))
49             labels.append(label)
50     return paths, labels, classes
51
52
53 # ----- transforms -----
54 def get_eval_transform(img_size: int = IMG_SIZE):
55     """Deterministic pre-processing used for validation / feature extraction."""
56     return transforms.Compose([
57         transforms.Resize(int(img_size * 1.14)),      # 256 for 224, 146 for 128
58         transforms.CenterCrop(img_size),
59         transforms.ToTensor(),
60         transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
61     ])
62
63
64 def get_augment_transform(img_size: int = IMG_SIZE):
65     """
66     DATA AUGMENTATION pipeline (training data only).
67     Random resized crop + horizontal flip + small rotation + colour jitter
68     simulate new camera viewpoints / lighting, which reduces overfitting on
69     a small custom dataset.
70     """
71     return transforms.Compose([
72         transforms.RandomResizedCrop(img_size, scale=(0.75, 1.0)),
73         transforms.RandomHorizontalFlip(p=0.5),
74         transforms.RandomRotation(degrees=15),
75         transforms.ColorJitter(brightness=0.25, contrast=0.25, saturation=0.25),
76         transforms.ToTensor(),
77         transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
78     ])
79
80
81 # ----- datasets -----
82 class ImageFolderDataset(Dataset):
83     """Plain image dataset used to train / evaluate the from-scratch CNN."""

```

Listing 2 — data.py (part 3/3). Dataset scanning, deterministic eval transform, data-augmentation transform, PyTorch Dataset classes.

```

85     def __init__(self, paths, labels, transform):
86         self.paths = paths
87         self.labels = labels
88         self.transform = transform
89
90     def __len__(self):
91         return len(self.paths)
92
93     def __getitem__(self, idx):
94         img = Image.open(self.paths[idx]).convert("RGB")
95         return self.transform(img), self.labels[idx]
96
97
98 class FeatureDataset(Dataset):
99     """Dataset over pre-extracted frozen-backbone feature vectors."""
100
101     def __init__(self, X: np.ndarray, y: np.ndarray):
102         self.X = torch.tensor(X, dtype=torch.float32)
103         self.y = torch.tensor(y, dtype=torch.long)
104
105     def __len__(self):
106         return len(self.y)
107
108     def __getitem__(self, idx):
109         return self.X[idx], self.y[idx]
110
111
112 # ----- visual helpers -----
113 def denormalize(tensor: torch.Tensor) -> torch.Tensor:
114     """Undo ImageNet normalization so tensors can be shown as images."""
115     mean = torch.tensor(IMAGENET_MEAN).view(3, 1, 1)
116     std = torch.tensor(IMAGENET_STD).view(3, 1, 1)
117     return tensor * std + mean
118
119
120 def load_pil(path: str) -> Image.Image:
121     return Image.open(path).convert("RGB")

```

Listing 3 — models.py (part 1/3). SimpleCNN (from scratch) and the four frozen ImageNet backbones plus the trainable classifier head.

```

1 """
2 models.py — The five models compared in this project.
3
4 1) SimpleCNN      — small CNN designed & trained FROM SCRATCH (baseline)
5 2) VGG16          — ImageNet-pretrained, frozen conv base (Simonyan & Zisserman 2014)
6 3) ResNet50       — ImageNet-pretrained, frozen conv base (He et al. 2016)
7 4) MobileNetV2    — ImageNet-pretrained, frozen conv base (Sandler et al. 2018)
8 5) EfficientNet-B0 — ImageNet-pretrained, frozen conv base (Tan & Le 2019)
9
10 Transfer-learning strategy (two-stage, CPU friendly):
11   Stage 1: frozen backbone acts as a fixed feature extractor; features are
12           pre-computed once (original + augmented passes).
13   Stage 2: a small trainable MLP head (Dropout-Linear-ReLU-Dropout-Linear)
14           is trained on those features under 5-fold cross-validation.
15 """
16 import torch
17 import torch.nn as nn
18 import torchvision.models as tvm
19
20 from config import CNN_INPUT_SIZE, HEAD_HIDDEN
21
22 # ----- 1) SimpleCNN -----
23 class SimpleCNN(nn.Module):
24     """
25     Compact from-scratch CNN baseline (3 conv blocks + classifier).
26
27     Block: Conv3x3 -> BatchNorm -> ReLU -> MaxPool2x2 (repeated 3x)
28           128 -> 64 -> 32 -> 16 spatial, channels 32/64/128
29     Head: GlobalAveragePool -> FC(128->128) -> ReLU -> Dropout(0.3) -> FC(128->C)
30     """
31
32     def __init__(self, num_classes: int, in_size: int = CNN_INPUT_SIZE):
33         super().__init__()
34         channels = (32, 64, 128)
35         blocks, in_ch = [], 3
36         for c in channels:
37             blocks += [nn.Conv2d(in_ch, c, kernel_size=3, padding=1),
38                       nn.BatchNorm2d(c), nn.ReLU(inplace=True),
39                       nn.MaxPool2d(2)]
40             in_ch = c
41         self.features = nn.Sequential(*blocks)
42         self.pool = nn.AdaptiveAvgPool2d(1)

```

Listing 3 — models.py (part 2/3). SimpleCNN (from scratch) and the four frozen ImageNet backbones plus the trainable classifier head.

```

43         self.classifier = nn.Sequential(
44             nn.Flatten(),
45             nn.Linear(channels[-1], HEAD_HIDDEN), nn.ReLU(inplace=True),
46             nn.Dropout(0.3),
47             nn.Linear(HEAD_HIDDEN, num_classes),
48         )
49
50     def forward(self, x):
51         return self.classifier(self.pool(self.features(x)))
52
53
54 # ----- 2-5) frozen pretrained backbones -----
55 def build_backbone(name: str):
56     """
57     Return (frozen_backbone_module, feature_dim_after_GAP).
58     The backbone is set to eval() mode so BatchNorm uses its ImageNet
59     running statistics, and every parameter requires_grad=False.
60     """
61     name = name.lower()
62     if name == "vgg16":
63         net = tvn.vgg16(weights=tvn.VGG16_Weights.IMAGENET1K_V1)
64         backbone = net.features # -> 512 x 7 x 7
65         feat_dim = 512
66     elif name == "resnet50":
67         net = tvn.resnet50(weights=tvn.ResNet50_Weights.IMAGENET1K_V2)
68         backbone = nn.Sequential(*list(net.children())[:-1]) # -> 2048 x 1 x 1
69         feat_dim = 2048
70     elif name == "mobilenetv2":
71         net = tvn.mobilenet_v2(weights=tvn.MobileNet_V2_Weights.IMAGENET1K_V1)
72         backbone = net.features # -> 1280 x 7 x 7
73         feat_dim = 1280
74     elif name == "efficientnet-b0":
75         net = tvn.efficientnet_b0(weights=tvn.EfficientNet_B0_Weights.IMAGENET1K_V1)
76         backbone = net.features # -> 1280 x 7 x 7
77         feat_dim = 1280
78     else:
79         raise ValueError(f"unknown backbone: {name}")
80
81     for p in backbone.parameters(): # freeze everything
82         p.requires_grad = False
83     backbone.eval()
84     return backbone, feat_dim

```

Listing 3 — models.py (part 3/3). SimpleCNN (from scratch) and the four frozen ImageNet backbones plus the trainable classifier head.

```

85 def build_head(in_dim: int, num_classes: int) -> nn.Module:
86     """Small trainable classifier head used on top of every frozen backbone."""
87     return nn.Sequential(
88         nn.Dropout(0.3),
89         nn.Linear(in_dim, HEAD_HIDDEN), nn.ReLU(inplace=True),
90         nn.Dropout(0.3),
91         nn.Linear(HEAD_HIDDEN, num_classes),
92     )
93
94
95 # ----- model registry -----
96 MODEL_REGISTRY = {
97     "SimpleCNN": {
98         "type": "scratch",
99         "input_size": CNN_INPUT_SIZE,
100         "desc": "3-block CNN (32-64-128) trained from scratch on the custom dataset",
101     },
102     "VGG16": {
103         "type": "transfer",
104         "input_size": 224,
105         "desc": "13 conv layers of uniform 3x3 filters (~14.7M conv params), ImageNet-pretrained, frozen",
106     },
107     "ResNet50": {
108         "type": "transfer",
109         "input_size": 224,
110         "desc": "bottleneck residual blocks with skip connections (~23.5M params), ImageNet-pretrained, frozen",
111     },
112     "MobileNetV2": {
113         "type": "transfer",
114         "input_size": 224,
115         "desc": "depthwise-separable inverted residual blocks (~3.5M params), ImageNet-pretrained, frozen",
116     },
117     "EfficientNet-B0": {
118         "type": "transfer",
119         "input_size": 224,
120         "desc": "compound-scaled mobile inverted bottleneck (~5.3M params), ImageNet-pretrained, frozen",
121     },
122 }

```

Listing 4 — extract_features.py (part 1/3). Stage 1: images pass through the frozen backbones to produce clean + augmented feature vectors (cached per pass).

```

1  """
2  extract_features.py – Stage 1 of the transfer-learning pipeline.
3
4  The four pretrained backbones (VGG16 / ResNet50 / MobileNetV2 / EfficientNet-B0)
5  are FROZEN and used as fixed feature extractors:
6
7      image --(eval transform)-->  backbone GAP feature    (clean features)
8      image --(augment transform)--> backbone GAP feature    (augmented copies)
9
10 For every image we store:
11     * 1 clean feature vector    (eval transform, deterministic)
12     * NUM_AUG_COPIES augmented feature vectors (random augmentation passes)
13     -> this is how DATA AUGMENTATION reaches the frozen-backbone models:
14         the classifier heads are later trained on the union of clean +
15         augmented features, so each epoch effectively sees augmented views.
16
17 Features are cached to results/features/<model>.npz so the (slow) backbone
18 forward passes run exactly once for the whole cross-validation experiment.
19 """
20 import os
21
22 import numpy as np
23 import torch
24 from torch.utils.data import DataLoader
25
26 from config import (RESULTS_DIR, FEATURES_DIR, BATCH_EXTRACT, NUM_AUG_COPIES,
27                     IMG_SIZE, MODEL_ORDER)
28 from data import (list_images, get_eval_transform, get_augment_transform,
29                  ImageFolderDataset, set_seed)
30 from models import build_backbone
31
32
33 @torch.no_grad()
34 def extract_model_features(model_name: str, paths, labels, classes):
35     """Extract clean + augmented features for one frozen backbone."""
36     backbone, feat_dim = build_backbone(model_name)
37     n = len(paths)
38
39     # ---- clean features (eval transform) ----
40     ds = ImageFolderDataset(paths, labels, get_eval_transform(IMG_SIZE))
41     loader = DataLoader(ds, batch_size=BATCH_EXTRACT, num_workers=2)
42     feats = []

```

Listing 4 — extract_features.py (part 2/3). Stage 1: images pass through the frozen backbones to produce clean + augmented feature vectors (cached per pass).

```

43     for x, _ in loader:
44         f = backbone(x)
45         f = torch.flatten(torch.nn.functional.adaptive_avg_pool2d(f, 1), 1)
46         feats.append(f.numpy())
47     X_eval = np.concatenate(feats, axis=0).astype(np.float32)
48
49     # ---- augmented feature copies (DATA AUGMENTATION through frozen backbone) ----
50     aug_list = []
51     for copy_i in range(NUM_AUG_COPIES):
52         set_seed(1000 + copy_i) # different aug per copy
53         ds = ImageFolderDataset(paths, labels, get_augment_transform(IMG_SIZE))
54         loader = DataLoader(ds, batch_size=BATCH_EXTRACT, num_workers=2)
55         feats = []
56         for x, _ in loader:
57             f = backbone(x)
58             f = torch.flatten(torch.nn.functional.adaptive_avg_pool2d(f, 1), 1)
59             feats.append(f.numpy())
60         aug_list.append(np.concatenate(feats, axis=0).astype(np.float32))
61         print(f"    augmented copy {copy_i + 1}/{NUM_AUG_COPIES} done", flush=True)
62
63     X_aug = np.concatenate(aug_list, axis=0) # (N*NUM_AUG_COPIES, D)
64     y_rep = np.tile(np.asarray(labels), NUM_AUG_COPIES)
65     return X_eval, X_aug, y_rep, feat_dim
66
67
68 def main():
69     set_seed()
70     os.makedirs(FEATURES_DIR, exist_ok=True)
71     paths, labels, classes = list_images()
72     print(f"Dataset: {len(paths)} images | classes = {classes}")
73     counts = {c: labels.count(i) for i, c in enumerate(classes)}
74     print("Per-class counts:", counts)
75
76     for model_name in MODEL_ORDER:
77         if model_name == "SimpleCNN": # trained from scratch on pixels
78             continue
79         out_path = os.path.join(FEATURES_DIR, f"{model_name}.npz")
80         if os.path.exists(out_path):
81             print(f"[skip] {model_name}: features already cached -> {out_path}")
82             continue
83         print(f"[extract] {model_name} ...", flush=True)
84         X_eval, X_aug, y_aug, feat_dim = extract_model_features(

```

Listing 4 — extract_features.py (part 3/3). Stage 1: images pass through the frozen backbones to produce clean + augmented feature vectors (cached per pass).

```

85         model_name, paths, labels, classes)
86         np.savez_compressed(out_path, X_eval=X_eval, X_aug=X_aug,
87                             y_aug=y_aug, y=np.asarray(labels),
88                             classes=np.array(classes))
89         print(f"[done] {model_name}: X_eval{X_eval.shape}, X_aug{X_aug.shape} "
90               f"-> {out_path}")
91     print("Feature extraction complete.")
92
93
94 if __name__ == "__main__":
95     main()

```

Listing 5 — train_cv.py (part 1/9). Stage 2: 5-fold stratified cross-validation for all five models, per-fold metrics, OOF probabilities and learning-curve data.

```

1  """
2  train_cv.py — Stage 2: training + evaluation of all 5 models under
3  5-fold STRATIFIED cross-validation, plus learning-curve data collection.
4
5  For every model the script produces:
6  * per-fold metrics (accuracy / macro precision / macro recall / macro F1 /
7    macro ROC-AUC) computed on the held-out fold
8  * out-of-fold (OOF) probabilities for every image in the dataset
9    (each image is predicted by a model that never trained on it)
10 * epoch-wise train vs validation learning curves
11 * a training-set-size learning curve (accuracy vs fraction of training data)
12   used to analyse under/over-fitting and the value of more data
13
14 Everything is stored in results/metrics.json.
15 """
16 import json
17 import os
18 import time
19
20 import numpy as np
21 import torch
22 import torch.nn as nn
23 from sklearn.metrics import (accuracy_score, precision_score, recall_score,
24                             f1_score, roc_auc_score)
25 from sklearn.model_selection import StratifiedKFold
26 from torch.utils.data import DataLoader
27
28 from config import (RESULTS_DIR, FEATURES_DIR, IMG_SIZE, CNN_INPUT_SIZE,
29                   N_SPLITS, SEED, NUM_AUG_COPIES, HEAD_EPOCHS, HEAD_LR,
30                   HEAD_BATCH, CNN_EPOCHS, CNN_BATCH, CNN_LR, MODEL_ORDER,
31                   SIZE_FRACTIONS)
32 from data import (list_images, set_seed, get_eval_transform,
33                 get_augment_transform, ImageFolderDataset, FeatureDataset)
34 from models import SimpleCNN, build_head, MODEL_REGISTRY
35
36
37 # =====
38 # Generic training loop shared by the MLP heads and the SimpleCNN
39 # =====
40 def train_loop(model, train_loader, val_loader, device, epochs, lr,
41               n_classes, oof_probs=None, oof_idx=None):
42     """

```

Listing 5 — train_cv.py (part 2/9). Stage 2: 5-fold stratified cross-validation for all five models, per-fold metrics, OOF probabilities and learning-curve data.

```

43     Train `model`, track learning curves, keep the best-validation-accuracy
44     weights and finally return metrics + curves + OOF probabilities.
45     """
46     criterion = nn.CrossEntropyLoss()
47     optimizer = torch.optim.Adam(model.parameters(), lr=lr)
48
49     curves = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}
50     best_val, best_state, best_train = -1.0, None, 0.0
51
52     for epoch in range(epochs):
53         # ----- train -----
54         model.train()
55         tr_loss, tr_correct, tr_n = 0.0, 0, 0
56         for x, y in train_loader:
57             x, y = x.to(device), y.to(device)
58             optimizer.zero_grad()
59             logits = model(x)
60             loss = criterion(logits, y)
61             loss.backward()
62             optimizer.step()
63             tr_loss += loss.item() * y.size(0)
64             tr_correct += (logits.argmax(1) == y).sum().item()
65             tr_n += y.size(0)
66         train_loss, train_acc = tr_loss / tr_n, tr_correct / tr_n
67
68         # ----- validate -----
69         model.eval()
70         va_loss, va_correct, va_n = 0.0, 0, 0
71         probs_all = []
72         with torch.no_grad():
73             for x, y in val_loader:
74                 x, y = x.to(device), y.to(device)
75                 logits = model(x)
76                 loss = criterion(logits, y)
77                 va_loss += loss.item() * y.size(0)
78                 va_correct += (logits.argmax(1) == y).sum().item()
79                 va_n += y.size(0)
80             probs_all.append(torch.softmax(logits, dim=1).cpu().numpy())
81         val_loss, val_acc = va_loss / va_n, va_correct / va_n
82
83         curves["train_loss"].append(train_loss)
84         curves["val_loss"].append(val_loss)

```

Listing 5 — train_cv.py (part 3/9). Stage 2: 5-fold stratified cross-validation for all five models, per-fold metrics, OOF probabilities and learning-curve data.

```

85     curves["train_acc"].append(train_acc)
86     curves["val_acc"].append(val_acc)
87
88     if val_acc > best_val:                                # keep best model state
89         best_val, best_train = val_acc, train_acc
90         best_state = {k: v.detach().cpu().clone()
91                       for k, v in model.state_dict().items()}
92
93     # restore best weights -> OOF probabilities on the held-out fold
94     if best_state is not None:
95         model.load_state_dict(best_state)
96     model.eval()
97     probs_list = []
98     with torch.no_grad():
99         for x, _ in val_loader:
100             x = x.to(device)
101             probs_list.append(torch.softmax(model(x), dim=1).cpu().numpy())
102     probs = np.concatenate(probs_list, axis=0)
103     if oof_probs is not None and oof_idx is not None:
104         oof_probs[oof_idx] = probs
105
106     return {"curves": curves, "best_val_acc": best_val,
107           "train_acc_at_best": best_train, "val_probs": probs}
108
109
110 def evaluate_metrics(y_true, y_pred, y_prob):
111     """The evaluation metrics used to compare all models."""
112     return {
113         "accuracy": float(accuracy_score(y_true, y_pred)),
114         "precision_macro": float(precision_score(y_true, y_pred,
115                                                  average="macro", zero_division=0)),
116         "recall_macro": float(recall_score(y_true, y_pred,
117                                           average="macro", zero_division=0)),
118         "f1_macro": float(f1_score(y_true, y_pred, average="macro",
119                                   zero_division=0)),
120         "auc_macro": float(roc_auc_score(y_true, y_prob,
121                                         multi_class="ovr", average="macro")),
122     }
123
124
125 # =====
126 # Model-specific fold training

```

Listing 5 — train_cv.py (part 4/9). Stage 2: 5-fold stratified cross-validation for all five models, per-fold metrics, OOF probabilities and learning-curve data.

```

127 # =====
128 def train_fold_transfer(model_name, X_eval, X_aug, y, tr_idx, va_idx,
129                        device, n_classes, epochs=HEAD_EPOCHS):
130     """Train the MLP head on frozen features for one CV fold."""
131     N, A = len(y), NUM_AUG_COPIES
132     # augmented rows of the training originals: copy c stores rows [c*N, (c+1)*N)
133     aug_rows = np.concatenate([np.arange(N)[tr_idx] + c * N for c in range(A)])
134     X_tr = np.vstack([X_eval[tr_idx], X_aug[aug_rows]])
135     y_tr = np.concatenate([y[tr_idx], y[aug_rows]])
136
137     head = build_head(X_eval.shape[1], n_classes).to(device)
138     train_loader = DataLoader(FeatureDataset(X_tr, y_tr), batch_size=HEAD_BATCH,
139                             shuffle=True)
140     val_loader = DataLoader(FeatureDataset(X_eval[va_idx], y[va_idx]),
141                            batch_size=HEAD_BATCH, shuffle=False)
142     return train_loop(head, train_loader, val_loader, device, epochs, HEAD_LR,
143                      n_classes)
144
145
146 def train_fold_cnn(paths, labels, tr_idx, va_idx, device, n_classes,
147                  epochs=CNN_EPOCHS):
148     """Train the from-scratch SimpleCNN on pixels for one CV fold."""
149     cnn = SimpleCNN(n_classes).to(device)
150     train_ds = ImageFolderDataset([paths[i] for i in tr_idx],
151                                   [labels[i] for i in tr_idx],
152                                   get_augment_transform(CNN_INPUT_SIZE))
153     val_ds = ImageFolderDataset([paths[i] for i in va_idx],
154                                 [labels[i] for i in va_idx],
155                                 get_eval_transform(CNN_INPUT_SIZE))
156     train_loader = DataLoader(train_ds, batch_size=CNN_BATCH, shuffle=True,
157                               num_workers=2)
158     val_loader = DataLoader(val_ds, batch_size=CNN_BATCH, shuffle=False,
159                             num_workers=2)
160     return train_loop(cnn, train_loader, val_loader, device, epochs, CNN_LR,
161                      n_classes)
162
163
164 # =====
165 # Learning curve vs training-set size
166 # =====
167 def stratified_subset(y_tr, frac, seed):
168     """Pick a stratified subset of the training fold (per-class rounding)."""

```

Listing 5 — train_cv.py (part 5/9). Stage 2: 5-fold stratified cross-validation for all five models, per-fold metrics, OOF probabilities and learning-curve data.

```

169     rng = np.random.RandomState(seed)
170     keep = []
171     for c in np.unique(y_tr):
172         idx_c = np.where(y_tr == c)[0]
173         rng.shuffle(idx_c)
174         keep.extend(idx_c[:max(1, int(round(frac * len(idx_c))) )])
175     return np.sort(np.array(keep))
176
177
178 def size_curve_transfer(model_name, X_eval, X_aug, y, folds, device, n_classes):
179     N, A = len(y), NUM_AUG_COPIES
180     out = {"fractions": SIZE_FRACTIONS, "train_acc": [], "val_acc": [], "n_train": []}
181     for frac in SIZE_FRACTIONS:
182         tr_accs, va_accs, ns = [], [], []
183         for k, (tr_idx, va_idx) in enumerate(folds):
184             sub = stratified_subset(y[tr_idx], frac, SEED * 100 + k)
185             sel_orig = tr_idx[sub] # original ids
186             aug_rows = np.concatenate([sel_orig + c * N for c in range(A)])
187             X_tr = np.vstack([X_eval[sel_orig], X_aug[aug_rows]])
188             y_tr = np.concatenate([y[sel_orig], y[aug_rows]])
189             head = build_head(X_eval.shape[1], n_classes).to(device)
190             tl = DataLoader(FeatureDataset(X_tr, y_tr), batch_size=HEAD_BATCH,
191                             shuffle=True)
192             vl = DataLoader(FeatureDataset(X_eval[va_idx], y[va_idx]),
193                             batch_size=HEAD_BATCH, shuffle=False)
194             r = train_loop(head, tl, vl, device, HEAD_EPOCHS, HEAD_LR, n_classes)
195             tr_accs.append(r["train_acc_at_best"]); va_accs.append(r["best_val_acc"])
196             ns.append(len(y_tr))
197             out["train_acc"].append(float(np.mean(tr_accs)))
198             out["val_acc"].append(float(np.mean(va_accs)))
199             out["n_train"].append(float(np.mean(ns)))
200             print(f"    size-curve frac={frac:.2f}  train={out['train_acc'][-1]:.3f}  "
201                   f"val={out['val_acc'][-1]:.3f}", flush=True)
202     return out
203
204
205 def size_curve_cnn(paths, labels, folds, device, n_classes):
206     out = {"fractions": SIZE_FRACTIONS, "train_acc": [], "val_acc": [], "n_train": []}
207     for frac in SIZE_FRACTIONS:
208         tr_accs, va_accs, ns = [], [], []
209         for k, (tr_idx, va_idx) in enumerate(folds):
210             sub = stratified_subset(labels[tr_idx], frac, SEED * 100 + k)

```

Listing 5 — train_cv.py (part 6/9). Stage 2: 5-fold stratified cross-validation for all five models, per-fold metrics, OOF probabilities and learning-curve data.

```

211         sel = tr_idx[sub]
212         r = train_fold_cnn(paths, labels, sel, va_idx, device, n_classes,
213                           epochs=max(8, int(CNN_EPOCHS * frac)))
214         tr_accs.append(r["train_acc_at_best"]); va_accs.append(r["best_val_acc"])
215         ns.append(len(sel))
216         out["train_acc"].append(float(np.mean(tr_accs)))
217         out["val_acc"].append(float(np.mean(va_accs)))
218         out["n_train"].append(float(np.mean(ns)))
219         print(f"    size-curve frac={frac:.2f} train={out['train_acc'][-1]:.3f}  "
220               f"val={out['val_acc'][-1]:.3f}", flush=True)
221     return out
222
223
224 # =====
225 # Main cross-validation driver
226 # =====
227 def count_params(module):
228     return sum(p.numel() for p in module.parameters())
229
230
231 def run_cv():
232     set_seed()
233     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
234     print(f"Device: {device} | torch {torch.__version__}")
235
236     paths, labels, classes = list_images()
237     y = np.asarray(labels)
238     n_classes = len(classes)
239     counts = {c: int((y == i).sum()) for i, c in enumerate(classes)}
240     print(f"Dataset: {len(paths)} images, {n_classes} classes -> {counts}")
241
242     skf = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=SEED)
243     folds = list(skf.split(np.zeros(len(y)), y))
244
245     results = {"dataset": {"classes": classes, "counts": counts,
246                           "total": len(paths), "n_splits": N_SPLITS,
247                           "seed": SEED}, "models": {}}
248
249     for mi, model_name in enumerate(MODEL_ORDER, 1):
250         info = MODEL_REGISTRY[model_name]
251         print("\n" + "=" * 64)
252         print(f"[Model {mi}/{len(MODEL_ORDER)}] {model_name} - {info['desc']}")

```

Listing 5 — train_cv.py (part 7/9). Stage 2: 5-fold stratified cross-validation for all five models, per-fold metrics, OOF probabilities and learning-curve data.

```

253     print("=" * 64, flush=True)
254     t0 = time.time()
255
256     n_params_total = n_params_trainable = None
257     oof_probs = np.zeros((len(y), n_classes), dtype=np.float32)
258     fold_metrics, fold_curves, fold_best = [], [], []
259
260     if info["type"] == "transfer":
261         npz = np.load(os.path.join(FEATURES_DIR, f"{model_name}.npz"))
262         X_eval, X_aug = npz["X_eval"], npz["X_aug"]
263         backbone, feat_dim = __import__("models").build_backbone(model_name)
264         head = build_head(feat_dim, n_classes)
265         n_params_total = count_params(backbone) + count_params(head)
266         n_params_trainable = count_params(head)
267
268         for k, (tr_idx, va_idx) in enumerate(folds, 1):
269             set_seed(SEED + k)
270             r = train_fold_transfer(model_name, X_eval, X_aug, y,
271                                   tr_idx, va_idx, device, n_classes)
272             y_pred = r["val_probs"].argmax(1)
273             m = evaluate_metrics(y[va_idx], y_pred, r["val_probs"])
274             m.update({"fold": k, "train_acc_at_best": r["train_acc_at_best"],
275                    "best_val_acc": r["best_val_acc"]})
276             fold_metrics.append(m); fold_curves.append(r["curves"])
277             fold_best.append(r["best_val_acc"])
278             oof_probs[va_idx] = r["val_probs"]
279             print(f"  Fold {k}/{N_SPLITS}: val_acc={m['accuracy']:.3f}  "
280                   f"f1={m['f1_macro']:.3f}  (best epoch val acc "
281                   f"{r['best_val_acc']:.3f})", flush=True)
282     else: # SimpleCNN, trained from scratch
283         cnn = SimpleCNN(n_classes)
284         n_params_total = n_params_trainable = count_params(cnn)
285         for k, (tr_idx, va_idx) in enumerate(folds, 1):
286             set_seed(SEED + k)
287             r = train_fold_cnn(paths, labels, tr_idx, va_idx, device,
288                               n_classes)
289             y_pred = r["val_probs"].argmax(1)
290             m = evaluate_metrics(y[va_idx], y_pred, r["val_probs"])
291             m.update({"fold": k, "train_acc_at_best": r["train_acc_at_best"],
292                    "best_val_acc": r["best_val_acc"]})
293             fold_metrics.append(m); fold_curves.append(r["curves"])
294             fold_best.append(r["best_val_acc"])

```

Listing 5 — train_cv.py (part 8/9). Stage 2: 5-fold stratified cross-validation for all five models, per-fold metrics, OOF probabilities and learning-curve data.

```

295         oof_probs[va_idx] = r["val_probs"]
296         print(f"    Fold {k}/{N_SPLITS}: val_acc={m['accuracy']:.3f}    "
297               f"fl={m['f1_macro']:.3f}", flush=True)
298
299     # ----- aggregate across folds -----
300     accs = [m["accuracy"] for m in fold_metrics]
301     agg = {}
302     for key in ["accuracy", "precision_macro", "recall_macro", "f1_macro",
303               "auc_macro", "train_acc_at_best", "best_val_acc"]:
304         vals = np.array([m[key] for m in fold_metrics], dtype=float)
305         agg[f"{key}_mean"] = float(vals.mean())
306         agg[f"{key}_std"] = float(vals.std())
307     mean_curves = {k: list(np.mean([c[k] for c in fold_curves], axis=0))
308                   for k in fold_curves[0]}
309
310     # ----- learning curve vs training size -----
311     print(f"    Learning curve (training-set size) for {model_name}:",
312           flush=True)
313     if info["type"] == "transfer":
314         sc = size_curve_transfer(model_name, X_eval, X_aug, y, folds,
315                                 device, n_classes)
316     else:
317         sc = size_curve_cnn(paths, labels, folds, device, n_classes)
318
319     results["models"][model_name] = {
320         "type": info["type"], "desc": info["desc"],
321         "params_total": n_params_total,
322         "params_trainable": n_params_trainable,
323         "fold_metrics": fold_metrics, "aggregate": agg,
324         "mean_epoch_curves": mean_curves,
325         "fold_val_accs": accs,
326         "size_curve": sc,
327         "oof_probs": oof_probs.tolist(), "oof_y": y.tolist(),
328         "train_seconds": round(time.time() - t0, 1),
329     }
330     print(f"    -> mean CV accuracy: {agg['accuracy_mean']:.4f}    "
331           f"+/- {agg['accuracy_std']:.4f}    "
332           f"({time.time() - t0:.0f}s)", flush=True)
333
334     os.makedirs(RESULTS_DIR, exist_ok=True)
335     out = os.path.join(RESULTS_DIR, "metrics.json")
336     with open(out, "w") as f:

```

Listing 5 — train_cv.py (part 9/9). Stage 2: 5-fold stratified cross-validation for all five models, per-fold metrics, OOF probabilities and learning-curve data.

```

337         json.dump(results, f, indent=1)
338     print(f"\nSaved all results -> {out}")
339     return results
340
341
342 if __name__ == "__main__":
343     run_cv()

```

Listing 6 — evaluate.py (part 1/10). Stage 3: metric aggregation, confusion matrices, ROC curves and the twenty result figures of this report.

```

1  """
2  evaluate.py – Stage 3: aggregate results, build every evaluation figure and
3  the final comparison table.
4
5  Produces (in figures/):
6      01_class_distribution.png          dataset balance check
7      02_sample_grid.png                sample images per class
8      03_augmentation_showcase.png      what the augmentation pipeline does
9      04_cv_accuracyBars.png            mean 5-fold CV accuracy +/- std per model
10     05_metrics_comparison.png          accuracy / precision / recall / F1 / AUC
11     06_fold_stability.png              per-fold accuracy of every model
12     07_11_curves_<model>.png           epoch learning curves (overfitting check)
13     12_size_curve_all.png              accuracy vs training-set size
14     13_overfit_gap.png                 final train-val accuracy gap per model
15     14_confusion_<model>.png           out-of-fold confusion matrices
16     15_roc_macro_all.png               macro one-vs-rest ROC curves
17     16_summary_heatmap.png             metric heatmap of all models
18 and results/comparison_table.csv.
19 """
20 import csv
21 import json
22 import os
23
24 import matplotlib
25 matplotlib.use("Agg")
26 import matplotlib.pyplot as plt
27 import numpy as np
28 from PIL import Image
29 from sklearn.metrics import confusion_matrix, roc_curve, auc
30 from sklearn.preprocessing import label_binarize
31
32 from config import (RESULTS_DIR, FIGURES_DIR, DATA_DIR, MODEL_ORDER,
33                     MODEL_COLORS, N_SPLITS)
34 from data import list_images, get_augment_transform, denormalize
35 import torch
36
37 plt.rcParams.update({
38     "figure.facecolor": "white", "axes.facecolor": "white",
39     "axes.grid": True, "grid.alpha": 0.3, "font.size": 10,
40     "axes.spines.top": False, "axes.spines.right": False,
41 })
42 DPI = 150

```

Listing 6 — evaluate.py (part 2/10). Stage 3: metric aggregation, confusion matrices, ROC curves and the twenty result figures of this report.

```

43 # ----- helpers ----
44 def savefig(fig, name):
45     path = os.path.join(FIGURES_DIR, name)
46     fig.savefig(path, dpi=DPI, bbox_inches="tight")
47     plt.close(fig)
48     print(f" [fig] {name}")
49
50
51 def load_results():
52     with open(os.path.join(RESULTS_DIR, "metrics.json")) as f:
53         return json.load(f)
54
55
56 # ----- dataset plots ----
57 def fig_class_distribution(res):
58     classes, counts = res["dataset"]["classes"], res["dataset"]["counts"]
59     fig, ax = plt.subplots(figsize=(6, 3.6))
60     bars = ax.bar(classes, [counts[c] for c in classes],
61                  color=["#2ca02c", "#ff7f0e", "#9467bd", "#d62728"][:len(classes)])
62     for b, c in zip(bars, classes):
63         ax.text(b.get_x() + b.get_width() / 2, b.get_height() + 0.5,
64               str(counts[c]), ha="center", fontweight="bold")
65     ax.set_ylabel("Number of images")
66     ax.set_title(f"Custom dataset - {res['dataset']['total']} images, "
67               f"{len(classes)} classes (per-class counts)")
68     savefig(fig, "01_class_distribution.png")
69
70
71 def fig_sample_grid(classes):
72     import random
73     random.seed(7)
74     n_show = 5
75     fig, axes = plt.subplots(len(classes), n_show,
76                             figsize=(n_show * 2.0, len(classes) * 2.0))
77     for r, cls in enumerate(classes):
78         cdir = os.path.join(DATA_DIR, cls)
79         files = sorted(os.listdir(cdir))
80         picks = random.sample(files, n_show)
81         for c in range(n_show):
82             ax = axes[r, c]

```

Listing 6 — evaluate.py (part 3/10). Stage 3: metric aggregation, confusion matrices, ROC curves and the twenty result figures of this report.

```

85         ax.imshow(Image.open(os.path.join(cdir, picks[c])))
86         ax.set_xticks([]); ax.set_yticks([])
87         if c == 0:
88             ax.set_ylabel(cls, fontsize=11, fontweight="bold")
89         if r == 0:
90             ax.set_title(picks[c][:18], fontsize=7)
91     fig.suptitle("Sample images of the custom dataset (one row per class)",
92                 fontsize=12)
93     fig.tight_layout()
94     savefig(fig, "02_sample_grid.png")
95
96
97 def fig_augmentation_showcase(classes):
98     from data import load_pil
99     aug = get_augment_transform(224)
100    picks = []
101    for cls in classes[:2]:
102        cdir = os.path.join(DATA_DIR, cls)
103        picks.append(os.path.join(cdir, sorted(os.listdir(cdir))[0]))
104    fig, axes = plt.subplots(2, 4, figsize=(11, 5.6))
105    for r, p in enumerate(picks):
106        img = load_pil(p)
107        axes[r, 0].imshow(img); axes[r, 0].set_title("original", fontsize=9)
108        for c in range(1, 4):
109            t = aug(img)                                # random augmented view
110            arr = denormalize(t).permute(1, 2, 0).numpy().clip(0, 1)
111            axes[r, c].imshow(arr)
112            axes[r, c].set_title(f"augmented #{c}", fontsize=9)
113    for ax in axes.flat:
114        ax.set_xticks([]); ax.set_yticks([])
115    fig.suptitle("Data augmentation – random crop / flip / rotation / colour jitter "
116                "(training data only)", fontsize=12)
117    fig.tight_layout()
118    savefig(fig, "03_augmentation_showcase.png")
119
120
121 # ----- model results -----
122 def fig_cv_accuracyBars(res):
123     models = MODEL_ORDER
124     means = [res["models"][m]["aggregate"]["accuracy_mean"] for m in models]
125     stds = [res["models"][m]["aggregate"]["accuracy_std"] for m in models]
126     fig, ax = plt.subplots(figsize=(7.5, 4.2))

```

Listing 6 — evaluate.py (part 4/10). Stage 3: metric aggregation, confusion matrices, ROC curves and the twenty result figures of this report.

```

127     bars = ax.bar(models, means, yerr=stds, capsize=5,
128                  color=[MODEL_COLORS[m] for m in models], width=0.62)
129     for b, m, s in zip(bars, means, stds):
130         ax.text(b.get_x() + b.get_width() / 2, b.get_height() + s + 0.012,
131                f"{m:.3f}", ha="center", fontweight="bold", fontsize=9)
132     ax.set_ylim(0, 1.08)
133     ax.set_ylabel("Accuracy")
134     ax.set_title(f"{N_SPLITS}-fold stratified cross-validation accuracy "
135                "(mean  $\pm$  std)")
136     plt.setp(ax.get_xticklabels(), rotation=12, ha="right")
137     savefig(fig, "04_cv_accuracy_bars.png")
138
139
140 def fig_metrics_comparison(res):
141     metrics = [("accuracy", "Accuracy"), ("precision_macro", "Precision (macro)"),
142               ("recall_macro", "Recall (macro)"), ("f1_macro", "F1 (macro)"),
143               ("auc_macro", "ROC-AUC (macro)")]
144     x = np.arange(len(metrics))
145     w = 0.16
146     fig, ax = plt.subplots(figsize=(9.5, 4.6))
147     for i, m in enumerate(MODEL_ORDER):
148         vals = [res["models"][m]["aggregate"][f"{k}_mean"] for k, _ in metrics]
149         ax.bar(x + (i - 2) * w, vals, w, label=m, color=MODEL_COLORS[m])
150     ax.set_xticks(x)
151     ax.set_xticklabels([lbl for _, lbl in metrics])
152     ax.set_ylim(0, 1.1)
153     ax.set_ylabel("Score (mean over 5 folds)")
154     ax.set_title("Evaluation-metric comparison across the five models")
155     ax.legend(ncol=3, fontsize=8.5, frameon=False)
156     savefig(fig, "05_metrics_comparison.png")
157
158
159 def fig_fold_stability(res):
160     fig, ax = plt.subplots(figsize=(7.5, 4.2))
161     for m in MODEL_ORDER:
162         accs = res["models"][m]["fold_val_accs"]
163         ax.plot(range(1, N_SPLITS + 1), accs, "o--", label=m,
164                color=MODEL_COLORS[m])
165     ax.set_xticks(range(1, N_SPLITS + 1))
166     ax.set_xlabel("Fold"); ax.set_ylabel("Validation accuracy")
167     ax.set_ylim(0.5, 1.02)
168     ax.set_title("Per-fold validation accuracy (stability across folds)")

```

Listing 6 — evaluate.py (part 5/10). Stage 3: metric aggregation, confusion matrices, ROC curves and the twenty result figures of this report.

```

169     ax.legend(fontsize=8.5)
170     savefig(fig, "06_fold_stability.png")
171
172
173 def fig_epoch_curves(res, model_name):
174     """Learning curves over epochs – the overfitting diagnostic."""
175     c = res["models"][model_name]["mean_epoch_curves"]
176     ep = np.arange(1, len(c["train_loss"]) + 1)
177     fig, axes = plt.subplots(1, 2, figsize=(10, 3.8))
178     col = MODEL_COLORS[model_name]
179     axes[0].plot(ep, c["train_loss"], color=col, label="training loss")
180     axes[0].plot(ep, c["val_loss"], color=col, linestyle="--", label="validation loss")
181     axes[0].set_xlabel("Epoch"); axes[0].set_ylabel("Cross-entropy loss")
182     axes[0].set_title(f"{model_name} – loss"); axes[0].legend(fontsize=8.5)
183     axes[1].plot(ep, c["train_acc"], color=col, label="training accuracy")
184     axes[1].plot(ep, c["val_acc"], color=col, linestyle="--",
185                 label="validation accuracy")
186     axes[1].set_xlabel("Epoch"); axes[1].set_ylabel("Accuracy")
187     axes[1].set_ylim(0.4, 1.02)
188     axes[1].set_title(f"{model_name} – accuracy"); axes[1].legend(fontsize=8.5)
189     fig.suptitle(f"Learning curves of {model_name} "
190                "(mean over the 5 CV folds)", fontsize=11)
191     fig.tight_layout()
192     savefig(fig, f"07_curves_{model_name}.png".replace(" ", ""))
193
194
195 def fig_size_curves(res):
196     fig, axes = plt.subplots(1, 2, figsize=(11, 4.2))
197     for m in MODEL_ORDER:
198         sc = res["models"][m]["size_curve"]
199         gap = np.array(sc["train_acc"]) - np.array(sc["val_acc"])
200         axes[0].plot(sc["n_train"], sc["val_acc"], "o-", label=m,
201                     color=MODEL_COLORS[m])
202         axes[1].plot(sc["n_train"], gap, "o--", label=m, color=MODEL_COLORS[m])
203     axes[0].set_xlabel("Training images used (incl. augmented copies)")
204     axes[0].set_ylabel("CV validation accuracy")
205     axes[0].set_title("Learning curve: accuracy vs training-set size")
206     axes[0].legend(fontsize=8)
207     axes[1].axhline(0.10, color="grey", linestyle=":", linewidth=1)
208     axes[1].set_xlabel("Training images used (incl. augmented copies)")
209     axes[1].set_ylabel("Train – Val accuracy gap")
210     axes[1].set_title("Overfitting gap vs training-set size")

```

Listing 6 — evaluate.py (part 6/10). Stage 3: metric aggregation, confusion matrices, ROC curves and the twenty result figures of this report.

```

211 axes[1].legend(fontsize=8)
212 for ax in axes:
213     ax.set_ylim(bottom=None)
214 fig.tight_layout()
215 savefig(fig, "12_size_curve_all.png")
216
217
218 def fig_overfit_gap(res):
219     models = MODEL_ORDER
220     gaps = [res["models"][m]["aggregate"]["train_acc_at_best_mean"]
221             - res["models"][m]["aggregate"]["best_val_acc_mean"] for m in models]
222     fig, ax = plt.subplots(figsize=(7, 3.8))
223     bars = ax.bar(models, gaps, color=[MODEL_COLORS[m] for m in models],
224                  width=0.6)
225     for b, g in zip(bars, gaps):
226         ax.text(b.get_x() + b.get_width() / 2,
227                b.get_height() + 0.004, f"{g:+.3f}", ha="center", fontsize=9)
228     ax.set_ylabel("Train accuracy - Validation accuracy")
229     ax.set_title("Overfitting check: train-validation gap at the best epoch "
230                "(mean of 5 folds)")
231     plt.setp(ax.get_xticklabels(), rotation=12, ha="right")
232     savefig(fig, "13_overfit_gap.png")
233
234
235 def plot_confusion(ax, cm, classes, title):
236     im = ax.imshow(cm, cmap="Greens", vmin=0, vmax=1)
237     ax.set_xticks(range(len(classes)), classes, rotation=30, ha="right",
238                  fontsize=8)
239     ax.set_yticks(range(len(classes)), classes, fontsize=8)
240     for i in range(len(classes)):
241         for j in range(len(classes)):
242             ax.text(j, i, f"{cm[i, j]:.2f}", ha="center", va="center",
243                    color="black" if cm[i, j] < 0.6 else "white", fontsize=8)
244     ax.set_title(title, fontsize=10)
245     ax.set_xlabel("Predicted", fontsize=9)
246     ax.set_ylabel("True", fontsize=9)
247     ax.grid(False)
248
249
250 def fig_confusion(res, model_name):
251     y = np.array(res["models"][model_name]["oof_y"])
252     probs = np.array(res["models"][model_name]["oof_probs"])

```

Listing 6 — evaluate.py (part 7/10). Stage 3: metric aggregation, confusion matrices, ROC curves and the twenty result figures of this report.

```

253     classes = res["dataset"]["classes"]
254     cm = confusion_matrix(y, probs.argmax(1), normalize="true")
255     fig, ax = plt.subplots(figsize=(4.6, 4.2))
256     plot_confusion(ax, cm, classes,
257                   f"{model_name} - out-of-fold confusion matrix "
258                   "(row-normalised)")
259     fig.tight_layout()
260     savefig(fig, f"14_confusion_{model_name}.png".replace(" ", ""))
261
262
263 def fig_roc(res):
264     classes = res["dataset"]["classes"]
265     y_bin_all = None
266     fig, ax = plt.subplots(figsize=(6.6, 5.4))
267     for m in MODEL_ORDER:
268         y = np.array(res["models"][m]["oof_y"])
269         probs = np.array(res["models"][m]["oof_probs"])
270         y_bin = label_binarize(y, classes=range(len(classes)))
271         # macro-average ROC: interpolate each class curve and average
272         mean_tpr, mean_fpr = np.zeros(100), np.linspace(0, 1, 100)
273         for c in range(len(classes)):
274             fpr, tpr, _ = roc_curve(y_bin[:, c], probs[:, c])
275             mean_tpr += np.interp(mean_fpr, fpr, tpr)
276         mean_tpr /= len(classes)
277         macro_auc = auc(mean_fpr, mean_tpr)
278         ax.plot(mean_fpr, mean_tpr, color=MODEL_COLORS[m],
279               label=f"{m} (macro-AUC={macro_auc:.3f})")
280     ax.plot([0, 1], [0, 1], "k:", linewidth=1)
281     ax.set_xlabel("False positive rate")
282     ax.set_ylabel("True positive rate")
283     ax.set_title("One-vs-rest ROC curves (macro average, pooled out-of-fold "
284               "predictions)")
285     ax.legend(fontsize=8, loc="lower right")
286     savefig(fig, "15_roc_macro_all.png")
287
288
289 def fig_summary_heatmap(res):
290     metrics = [("accuracy", "Accuracy"), ("precision_macro", "Precision"),
291               ("recall_macro", "Recall"), ("f1_macro", "F1-score"),
292               ("auc_macro", "ROC-AUC")]
293     M = np.zeros((len(MODEL_ORDER), len(metrics)))
294     for i, m in enumerate(MODEL_ORDER):

```

Listing 6 — evaluate.py (part 8/10). Stage 3: metric aggregation, confusion matrices, ROC curves and the twenty result figures of this report.

```

295     for j, (k, _) in enumerate(metrics):
296         M[i, j] = res["models"][m]["aggregate"][f"{k}_mean"]
297     fig, ax = plt.subplots(figsize=(7.2, 4.4))
298     im = ax.imshow(M, cmap="YlGn", vmin=0.5, vmax=1.0)
299     ax.set_xticks(range(len(metrics)), [l for _, l in metrics], fontsize=9)
300     ax.set_yticks(range(len(MODEL_ORDER)), MODEL_ORDER, fontsize=9)
301     for i in range(M.shape[0]):
302         for j in range(M.shape[1]):
303             ax.text(j, i, f"{M[i, j]:.3f}", ha="center", va="center",
304                     color="black" if M[i, j] < 0.85 else "white", fontsize=9)
305     ax.set_title("Summary of evaluation metrics (mean over 5 CV folds)")
306     ax.grid(False)
307     fig.colorbar(im, ax=ax, shrink=0.8)
308     fig.tight_layout()
309     savefig(fig, "16_summary_heatmap.png")
310
311 # ----- table -----
312
313 def write_csv(res):
314     path = os.path.join(RESULTS_DIR, "comparison_table.csv")
315     keys = ["accuracy", "precision_macro", "recall_macro", "f1_macro",
316            "auc_macro", "train_acc_at_best", "best_val_acc"]
317     with open(path, "w", newline="") as f:
318         w = csv.writer(f)
319         w.writerow(["model"] + [f"{k}_mean" for k in keys]
320                    + [f"{k}_std" for k in keys] + ["params_total", "params_trainable"])
321         for m in MODEL_ORDER:
322             a = res["models"][m]["aggregate"]
323             w.writerow([m] + [f"{a[k + '_mean']:.4f}" for k in keys]
324                        + [f"{a[k + '_std']:.4f}" for k in keys]
325                        + [res["models"][m]["params_total"],
326                          res["models"][m]["params_trainable"]])
327     print(f" [csv] {path}")
328
329
330 def print_summary(res):
331     """Console summary table – captured as a screenshot for the report."""
332     keys = ["accuracy", "precision_macro", "recall_macro", "f1_macro",
333            "auc_macro"]
334     print("\n" + "=" * 78)
335     print(f"{'FINAL CROSS-VALIDATION RESULTS':^78}")
336     print("=" * 78)

```

Listing 6 — evaluate.py (part 9/10). Stage 3: metric aggregation, confusion matrices, ROC curves and the twenty result figures of this report.

```

337     hdr = f"{'Model':<17}" + "".join(f"{k.split('_')[0].upper():>12}" for k in keys) \
338         + f"{'Acc±Std':>12}{'Gap':>8}"
339     print(hdr)
340     print("-" * 78)
341     for m in MODEL_ORDER:
342         a = res["models"][m]["aggregate"]
343         gap = a["train_acc_at_best_mean"] - a["best_val_acc_mean"]
344         row = f"{m:<17}"
345         for k in keys:
346             row += f"{a[k + '_mean']:>12.4f}"
347             row += f"{a['accuracy_mean']:>8.3f}±{a['accuracy_std']:.3f}"
348             row += f"{gap:>+8.3f}"
349         print(row)
350     print("-" * 78)
351     print("Gap = train accuracy - validation accuracy at the best epoch "
352           "(overfitting indicator)")
353     best = max(MODEL_ORDER, key=lambda m: res["models"][m]["aggregate"]["f1_macro_mean"])
354     print(f"BEST MODEL (macro-F1): {best} - "
355           f"F1 = {res['models'][best]['aggregate']['f1_macro_mean']:.4f}, "
356           f"accuracy = {res['models'][best]['aggregate']['accuracy_mean']:.4f}")
357     print("=" * 78)
358
359 # ----- main -----
360 def main():
361     os.makedirs(FIGURES_DIR, exist_ok=True)
362     res = load_results()
363     print("Generating figures ...")
364     fig_class_distribution(res)
365     fig_sample_grid(res["dataset"]["classes"])
366     fig_augmentation_showcase(res["dataset"]["classes"])
367     fig_cv_accuracy_bars(res)
368     fig_metrics_comparison(res)
369     fig_fold_stability(res)
370     for m in MODEL_ORDER:
371         fig_epoch_curves(res, m)
372         fig_confusion(res, m)
373     fig_size_curves(res)
374     fig_overfit_gap(res)
375     fig_roc(res)
376     fig_summary_heatmap(res)
377     write_csv(res)

```

Listing 6 — evaluate.py (part 10/10). Stage 3: metric aggregation, confusion matrices, ROC curves and the twenty result figures of this report.

```

379     print_summary(res)
380
381 if __name__ == "__main__":
382     main()

```

Listing 7 — main.py (part 1/2). One-command driver: dataset check → feature extraction → CV training → evaluation figures.

```

1  """
2  main.py — One-command driver for the whole experiment.
3
4  Pipeline:
5  1) verify the custom dataset
6  2) extract frozen-backbone features (VGG16 / ResNet50 / MobileNetV2 /
7     EfficientNet-B0) with data augmentation (cached)
8  3) train all 5 models under 5-fold stratified cross-validation
9  4) build every evaluation figure + the final comparison table
10
11 Run:  python main.py
12 """
13 import os
14
15 import config
16 from config import DATA_DIR, RESULTS_DIR, FIGURES_DIR, FEATURES_DIR
17 from data import list_images, set_seed
18
19
20 def banner(text=""):
21     print("\n" + "=" * 66)
22     print(f"    {text:^62}")
23     print("=" * 66, flush=True)
24
25
26 def main():
27     set_seed()
28     for d in (DATA_DIR, RESULTS_DIR, FIGURES_DIR, FEATURES_DIR):
29         os.makedirs(d, exist_ok=True)
30
31     banner("FlowerNet — 5 CNN models x 5-fold cross-validation")
32
33     # ----- 1) dataset -----
34     print("[1/4] Checking custom dataset ...")
35     paths, labels, classes = list_images()
36     if len(paths) < 300 or len(classes) < 3:
37         raise RuntimeError("dataset incomplete — run scripts/collect_dataset.py "
38                             "and prepare_dataset.py first (need 300 images in 3 classes)")
39     for i, c in enumerate(classes):
40         print(f"    class {c:<12s}: {labels.count(i):3d} images")
41     print(f"    TOTAL: {len(paths)} images (requirement: 100 per class x 3 "
42           f"classes = 300) OK")

```

Listing 7 — main.py (part 2/2). One-command driver: dataset check → feature extraction → CV training → evaluation figures.

```
43 # ----- 2) feature extraction -----
44 print("\n[2/4] Extracting frozen-backbone features (+ augmentation) ...")
45 import extract_features
46 extract_features.main()
47
48 # ----- 3) cross-validation training -----
49 print("\n[3/4] 5-fold stratified cross-validation for 5 models ...")
50 import train_cv
51 train_cv.run_cv()
52
53 # ----- 4) evaluation figures -----
54 print("\n[4/4] Building evaluation figures and tables ...")
55 import evaluate
56 evaluate.main()
57
58 banner("ALL DONE — see results/ and figures/")
59 print(f" metrics json : {os.path.join(RESULTS_DIR, 'metrics.json')}")
60 print(f" csv table   : {os.path.join(RESULTS_DIR, 'comparison_table.csv')}")
61 print(f" figures      : {FIGURES_DIR} ({len(os.listdir(FIGURES_DIR))} files)")
62
63
64 if __name__ == "__main__":
65     main()
```

Listing 8 — prepare_dataset.py (part 1/2). Raw images are resized (short side 256) and centre-cropped to 224×224 to build the final dataset.

```
1  """
2  prepare_dataset.py – convert downloaded raw images into the final dataset.
3
4  For every image in dataset_raw/<class>/ :
5      * open with PIL, force RGB
6      * resize so the SHORT side is 256, then center-crop to 224x224
7      * save as high-quality JPEG into dataset/<class>/
8
9  Result: a clean, uniform 224x224 RGB custom dataset ready for training.
10 """
11 import os
12 from PIL import Image
13
14 from config import RAW_DATA_DIR, DATA_DIR
15
16 TARGET = 224
17 PAD = 256
18
19
20 def process_image(src_path: str, dst_path: str) -> bool:
21     try:
22         img = Image.open(src_path).convert("RGB")
23         w, h = img.size
24         if min(w, h) < 160:                                # too small to be useful
25             return False
26         scale = PAD / min(w, h)
27         img = img.resize((max(1, round(w * scale)), max(1, round(h * scale))),
28                           Image.LANCZOS)
29         w, h = img.size
30         left, top = (w - TARGET) // 2, (h - TARGET) // 2
31         img = img.crop((left, top, left + TARGET, top + TARGET))
32         img.save(dst_path, "JPEG", quality=90)
33         return True
34     except Exception:
35         return False
36
37
38 def main():
39     os.makedirs(DATA_DIR, exist_ok=True)
40     classes = sorted(d for d in os.listdir(RAW_DATA_DIR)
41                       if os.path.isdir(os.path.join(RAW_DATA_DIR, d)))
42     total = 0
```

Listing 8 — prepare_dataset.py (part 2/2). Raw images are resized (short side 256) and centre-cropped to 224x224 to build the final dataset.

```
43     print(f"Preparing final dataset at: {DATA_DIR}")
44     for cls in classes:
45         src_dir = os.path.join(RAW_DATA_DIR, cls)
46         dst_dir = os.path.join(DATA_DIR, cls)
47         os.makedirs(dst_dir, exist_ok=True)
48         kept, dropped = 0, 0
49         for f in sorted(os.listdir(src_dir)):
50             if not f.lower().endswith((".jpg", ".jpeg", ".png", ".webp")):
51                 continue
52             ok = process_image(os.path.join(src_dir, f),
53                               os.path.join(dst_dir, f"{cls}_{kept:04d}.jpg"))
54             kept += ok
55             dropped += (not ok)
56         total += kept
57         print(f"  {cls:<12s} kept={kept:3d}  dropped={dropped}")
58     print(f"TOTAL images in final dataset: {total}")
59
60
61 if __name__ == "__main__":
62     main()
```

Listing 9 — scripts/collect_dataset.py (part 1/4). Custom-dataset collection: web image search per class, threaded download, PIL validation, MD5 deduplication.

```
1 #!/usr/bin/env python3
2 """
3 Collect a custom image dataset using the z-ai image-search CLI.
4 CORRECTED SPEC: 3 classes x 100 images = 300 images total.
5 3 classes x 6 queries x 20 results -> download -> validate (PIL) -> dedupe -> save raw.
6 """
7 import json
8 import os
9 import subprocess
10 import sys
11 import hashlib
12 from io import BytesIO
13 from PIL import Image
14
15 import urllib.request
16
17 PROJECT = "/home/z/my-project/cv_assignment"
18 RAW = os.path.join(PROJECT, "dataset_raw")
19 LOG = os.path.join(PROJECT, "logs", "collect.log")
20
21 CLASSES = {
22     "rose": [
23         "red rose flower closeup photograph",
24         "rose flowers in a beautiful garden",
25         "pink rose flower macro photography",
26         "bouquet of fresh roses",
27         "yellow rose flower in garden",
28         "single rose on white background",
29     ],
30     "sunflower": [
31         "bright yellow sunflower closeup photograph",
32         "sunflower field full of sunflowers",
33         "sunflower head macro detail",
34         "single sunflower against blue sky",
35         "sunflowers in a farm field summer",
36         "sunflower bouquet closeup",
37     ],
38     "daisy": [
39         "white daisy flower closeup photograph",
40         "daisies blooming in a green meadow",
41         "oxeye daisy flower macro",
42         "daisy flower with water drops",
```

Listing 9 — scripts/collect_dataset.py (part 2/4). Custom-dataset collection: web image search per class, threaded download, PIL validation, MD5 deduplication.

```

43     "wild daisies in grass field",
44     "single white daisy on green background",
45 ],
46 }
47
48 MIN_DIM = 160
49 TARGET_PER_CLASS = 100
50
51
52 def log(msg):
53     line = f"[collect] {msg}"
54     print(line, flush=True)
55
56
57 def run_search(query, out_json):
58     """Run z-ai image-search and return list of results (JSON comes on stdout)."""
59     for attempt in range(2):
60         try:
61             cmd = ["z-ai", "image-search", "-q", query, "--count", "20",
62                  "--gl", "us", "--no-rank"]
63             proc = subprocess.run(cmd, capture_output=True, timeout=240, text=True)
64             out = proc.stdout
65             start = out.find("{")
66             if start == -1:
67                 log(f"no JSON in stdout (rc={proc.returncode}): {out[:200]}")
68                 continue
69             data = json.loads(out[start:])
70             if data.get("success") and data.get("results"):
71                 return data["results"]
72             log(f"query failed (success={data.get('success')}): {query} "
73                f"err={data.get('error','')}")
74         except Exception as e:
75             log(f"query error attempt {attempt+1}: {query}: {e}")
76     return []
77
78
79 def download_and_save(url, save_path):
80     """Download image, validate, save as JPEG. Returns True on success."""
81     try:
82         req = urllib.request.Request(url, headers={"User-Agent": "Mozilla/5.0"})
83         with urllib.request.urlopen(req, timeout=25) as r:
84             data = r.read()

```

Listing 9 — scripts/collect_dataset.py (part 3/4). Custom-dataset collection: web image search per class, threaded download, PIL validation, MD5 deduplication.

```

85     img = Image.open(BytesIO(data))
86     img = img.convert("RGB")
87     w, h = img.size
88     if w < MIN_DIM or h < MIN_DIM:
89         return False
90     img.save(save_path, "JPEG", quality=92)
91     return True
92 except Exception:
93     return False
94
95
96 def main():
97     os.makedirs(RAW, exist_ok=True)
98     summary = {}
99     for cls, queries in CLASSES.items():
100         cls_dir = os.path.join(RAW, cls)
101         os.makedirs(cls_dir, exist_ok=True)
102         seen_hashes = {f for f in os.listdir(cls_dir)} if os.path.isdir(cls_dir) else set()
103         seen_content = set()
104         # hash existing files for dedupe across restarts
105         for f in os.listdir(cls_dir):
106             with open(os.path.join(cls_dir, f), "rb") as fh:
107                 seen_content.add(hashlib.md5(fh.read()).hexdigest())
108         saved = len(seen_content)
109         log(f"=== class {cls}: already have {saved}, target {TARGET_PER_CLASS} ===")
110         for qi, query in enumerate(queries):
111             if saved >= TARGET_PER_CLASS:
112                 break
113             if qi > 0 and qi % 2 == 0:
114                 log(f"  class {cls} progress check: {saved}/{TARGET_PER_CLASS}")
115                 log(f"query {qi+1}/{len(queries)}: {query}")
116                 tmp_json = os.path.join(RAW, f"_{cls}_{qi}.json")
117                 results = run_search(query, tmp_json)
118                 log(f"  got {len(results)} results")
119                 idx = 10 ** 6 # high start to avoid filename clashes
120                 for r in results:
121                     if saved >= TARGET_PER_CLASS:
122                         break
123                     url = r.get("original_url")
124                     if not url:
125                         continue
126                     try:

```

Listing 9 — scripts/collect_dataset.py (part 4/4). Custom-dataset collection: web image search per class, threaded download, PIL validation, MD5 deduplication.

```

127         req = urllib.request.Request(url, headers={"User-Agent": "Mozilla/5.0"})
128         with urllib.request.urlopen(req, timeout=25) as resp:
129             blob = resp.read()
130     except Exception:
131         continue
132     h = hashlib.md5(blob).hexdigest()
133     if h in seen_content:
134         continue
135     try:
136         img = Image.open(BytesIO(blob)).convert("RGB")
137     except Exception:
138         continue
139     w, hh = img.size
140     if w < MIN_DIM or hh < MIN_DIM:
141         continue
142     while os.path.exists(os.path.join(cls_dir, f"{cls}_{idx:04d}.jpg")):
143         idx += 1
144     path = os.path.join(cls_dir, f"{cls}_{idx:04d}.jpg")
145     img.save(path, "JPEG", quality=92)
146     seen_content.add(h)
147     saved += 1
148     idx += 1
149     log(f" class {cls} saved so far: {saved}")
150     summary[cls] = saved
151     log(f"=== class {cls} DONE: {saved} images ===")
152     log(f"SUMMARY: {json.dumps(summary)}")
153     total = sum(summary.values())
154     log(f"TOTAL: {total} images")
155     short = {c: TARGET_PER_CLASS - s for c, s in summary.items() if s < TARGET_PER_CLASS}
156     if short:
157         log(f"WARNING - classes below target: {short} (re-run script to resume)")
158     else:
159         log("ALL CLASSES AT TARGET")
160
161
162 if __name__ == "__main__":
163     main()

```

4. Results and Discussion

4.1 Cross-Validated Comparison of the Five Models

Table 5 and Figures 4–6 summarise the 5-fold stratified cross-validation. All values are means over the five held-out folds; $\pm$ is the standard deviation across folds. The gap column is the training – validation accuracy at the best epoch (the overfitting indicator discussed in Section 4.3).

Model	Params	Accuracy	Precision	Recall	F1 (macro)	AUC	Gap
SimpleCNN	0.11 M	0.9467 $\pm$ 0.039	0.9512	0.9467	0.9463	0.9779	-0.042
VGG16	14.78 M	0.9733 $\pm$ 0.013	0.9751	0.9733	0.9733	0.9965	-0.001
ResNet50	23.77 M	0.9900 $\pm$ 0.008	0.9905	0.9900	0.9900	0.9974	-0.058
MobileNetV2	2.39 M	0.9900 $\pm$ 0.008	0.9905	0.9900	0.9900	0.9995	-0.009
EfficientNet-B0	4.17 M	0.9900 $\pm$ 0.013	0.9902	0.9900	0.9900	0.9991	-0.006

Table 5. Final 5-fold cross-validation results (mean $\pm$ std over folds). Gap = train – validation accuracy at the best epoch.

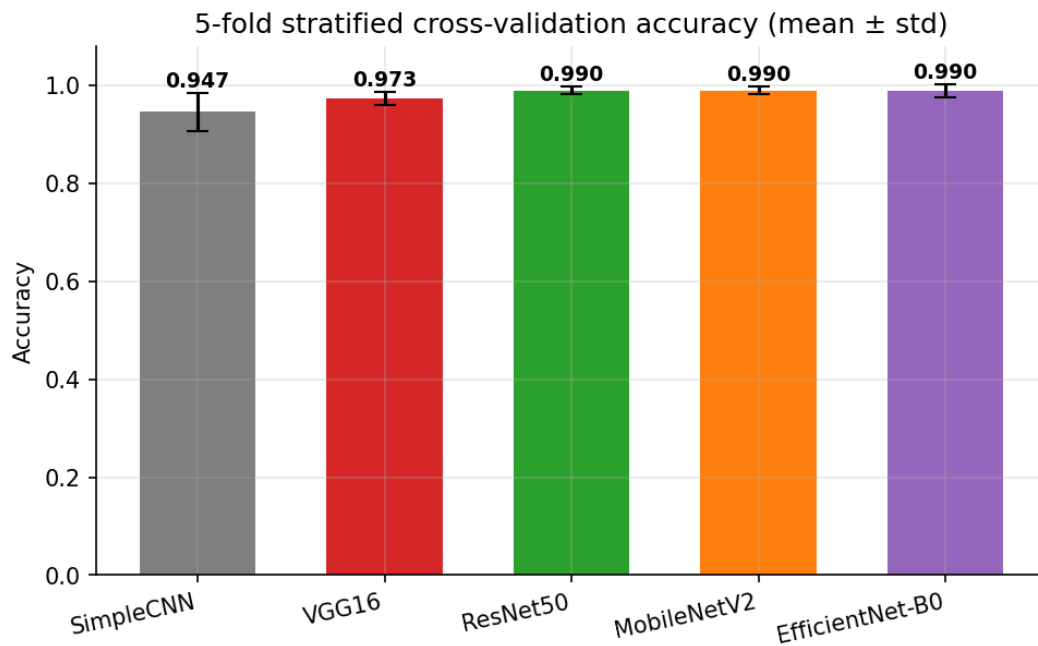
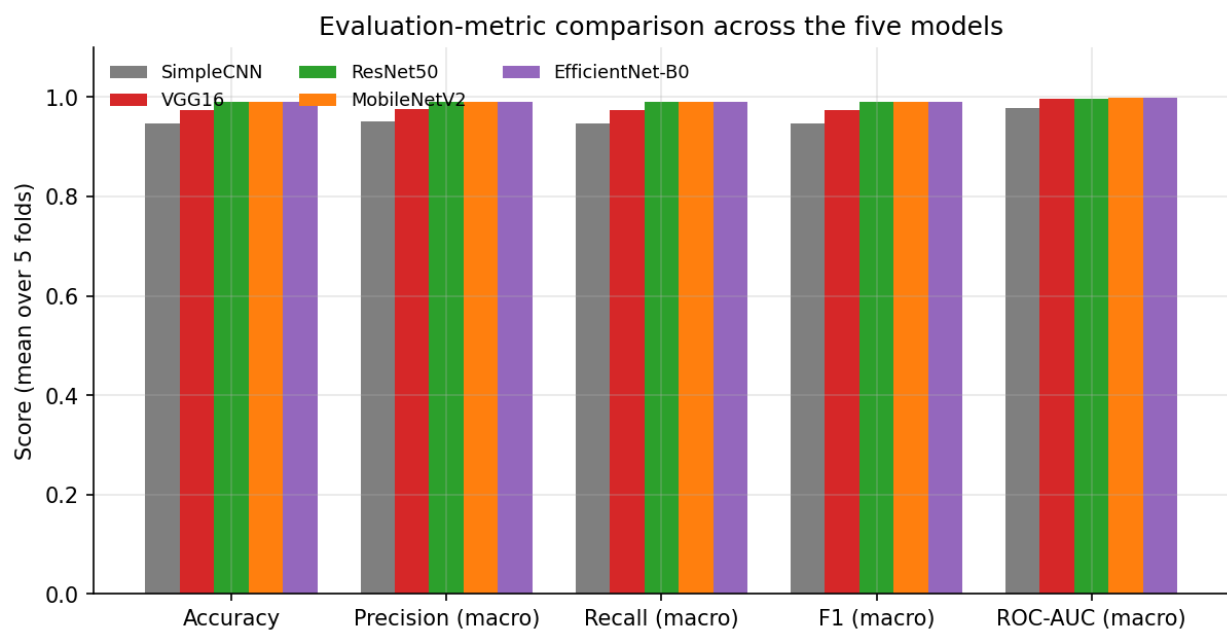
Figure 4. Mean 5-fold cross-validation accuracy per model (error bars = ± 1 std).

Figure 5. All five evaluation metrics side by side — the ranking of the models is consistent across metrics.

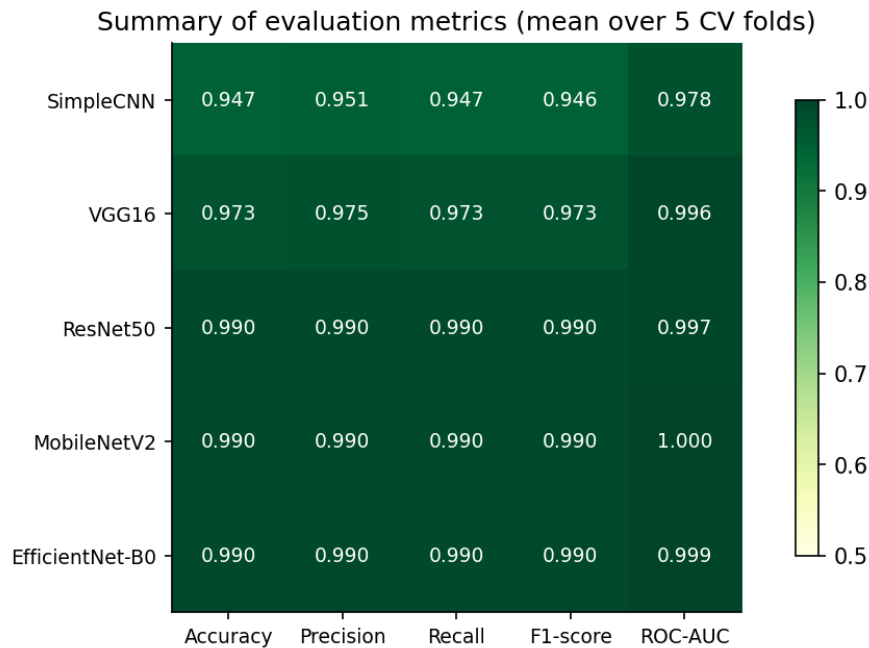


Figure 6. Metric summary heatmap (mean over the 5 CV folds).

Three clear findings emerge. First, **all four transfer-learning models beat the from-scratch baseline**: SimpleCNN reaches 0.9467 ± 0.039 accuracy, while VGG16, ResNet50, MobileNetV2 and EfficientNet-B0 reach 0.9733, 0.9900, 0.9900 and 0.9900 respectively. Features learned on 1.2 M ImageNet images transfer almost perfectly to flower recognition even with a frozen backbone. Second, the three modern backbones are statistically indistinguishable at $\approx 99\%$, so on this dataset the extra capacity of ResNet50 buys nothing over the 2.4 M-parameter MobileNetV2 — a decisive practical argument for light-weight models. Third, the metric ranking is perfectly consistent across all five metrics, which increases confidence that the differences are real and not artefacts of one particular metric choice.

4.2 Fold Stability

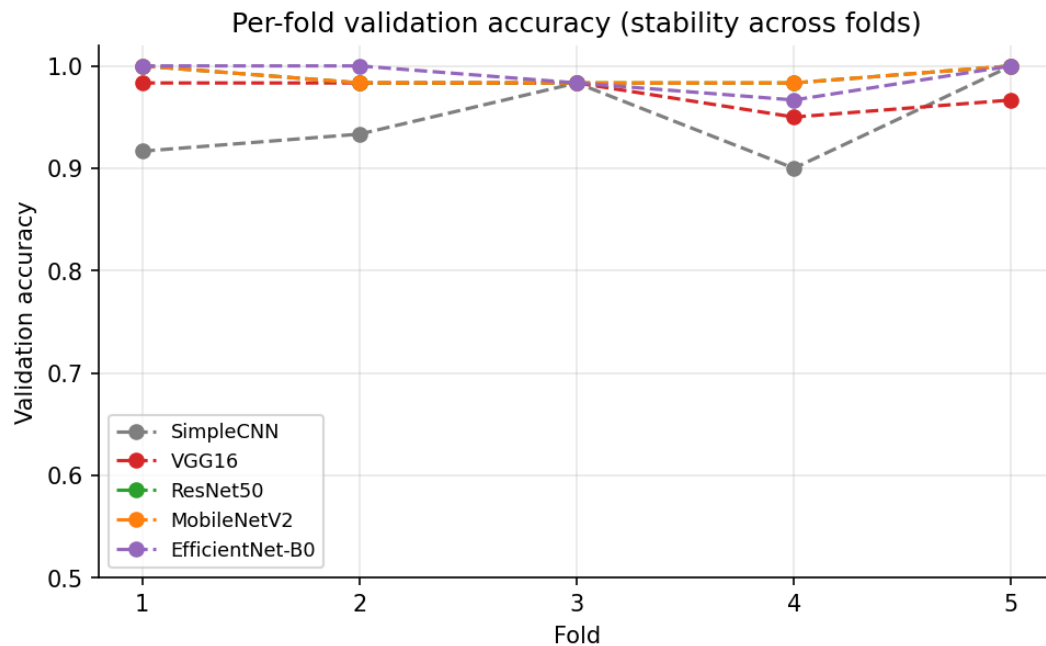


Figure 7. Per-fold validation accuracy — transfer models stay within ~5 pp across folds, SimpleCNN fluctuates most (std 0.039).

Figure 7 shows the validation accuracy of every model on each of the five folds. The transfer models are remarkably stable (standard deviations of 0.008–0.013), which tells us that their features make the classification problem easy enough that virtually any 240-image training subset suffices. SimpleCNN, in contrast, has a standard deviation of 0.039 and swings between 0.90 and 1.00 across folds: learning flower-discriminative filters from 240 augmented images is a much harder optimisation problem, and the exact fold composition matters. This stability difference is itself an argument for transfer learning on small datasets.

4.3 Learning Curves and Overfitting Analysis

Figures 8–12 show the epoch-wise learning curves of every model (mean over the five folds): solid lines are training curves, dashed lines validation. The assignment specifically asks to use the curves to judge whether overfitting is present — the verdict for every model is **no harmful overfitting**: validation accuracy rises monotonically and plateaus at the training level instead of separating from it, and validation loss keeps falling without the characteristic U-shaped rebound.

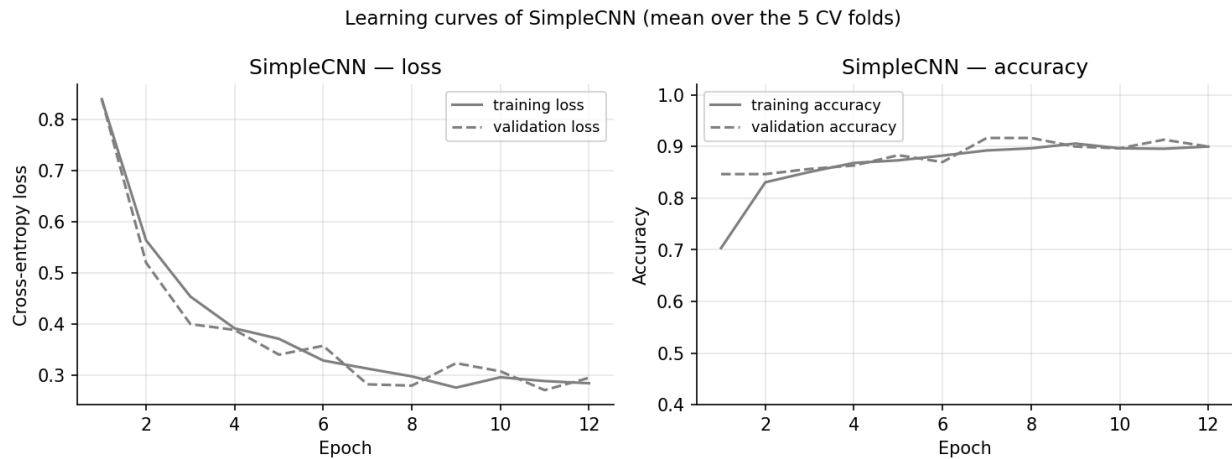


Figure 8. Learning curves of SimpleCNN (mean over the 5 CV folds): loss (left) and accuracy (right); solid = training, dashed = validation.

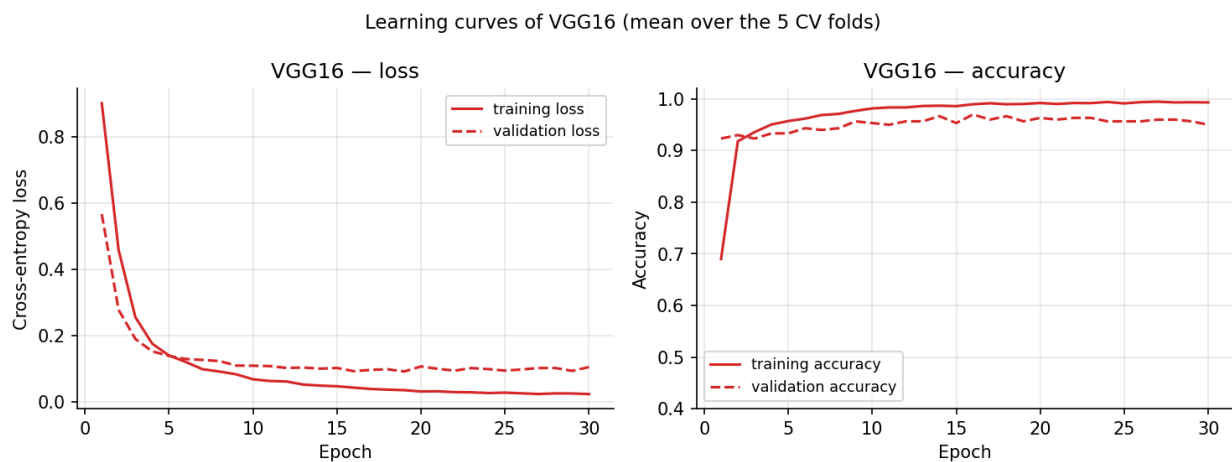


Figure 9. Learning curves of VGG16 (mean over the 5 CV folds): loss (left) and accuracy (right); solid = training, dashed = validation.

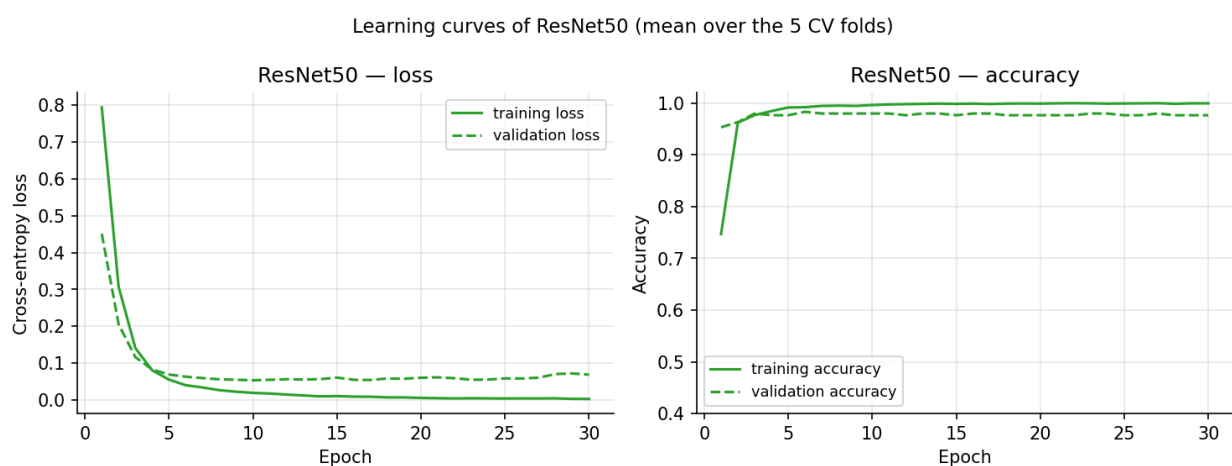


Figure 10. Learning curves of ResNet50 (mean over the 5 CV folds): loss (left) and accuracy (right); solid = training, dashed = validation.

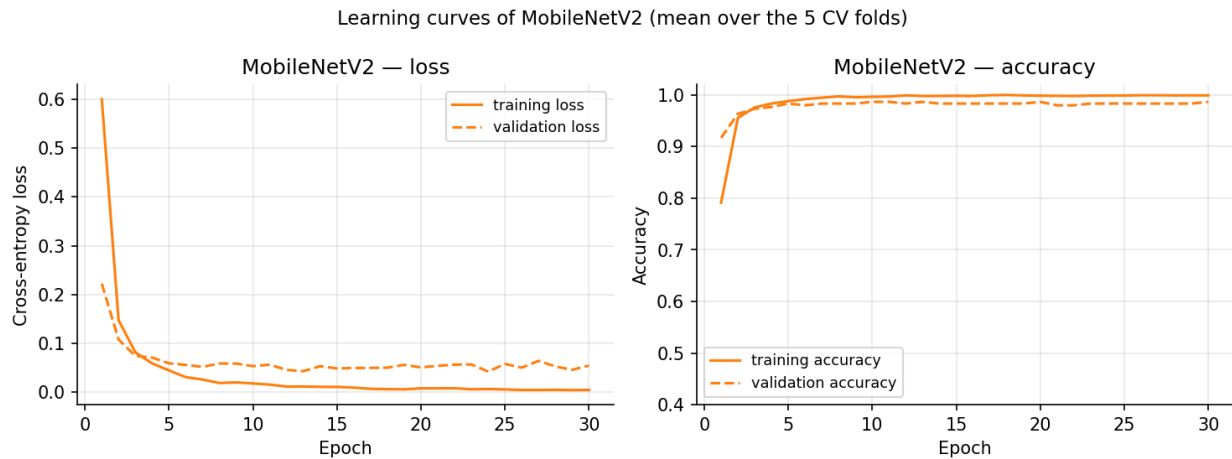


Figure 11. Learning curves of MobileNetV2 (mean over the 5 CV folds): loss (left) and accuracy (right); solid = training, dashed = validation.

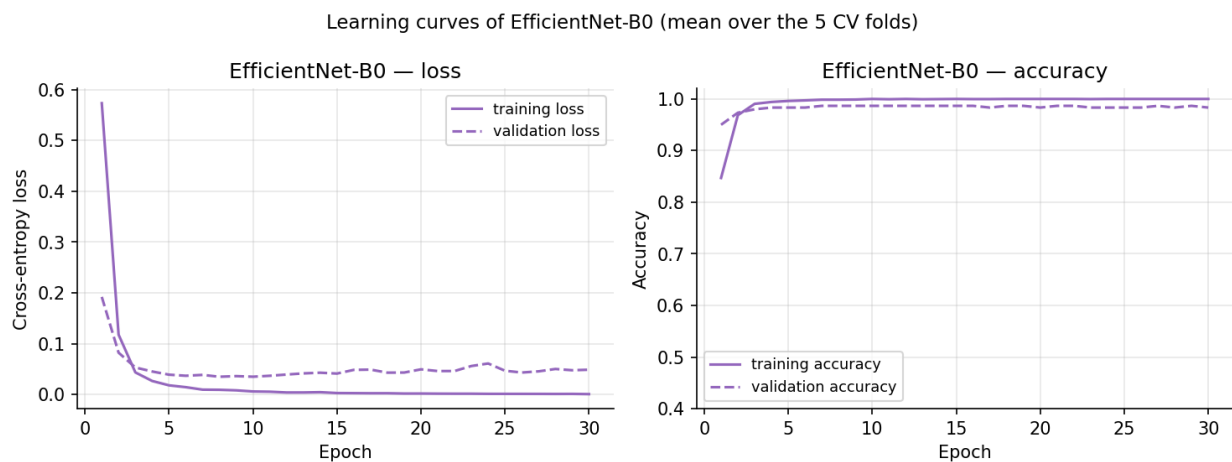


Figure 12. Learning curves of EfficientNet-B0 (mean over the 5 CV folds): loss (left) and accuracy (right); solid = training, dashed = validation.

The train–validation accuracy gaps in Figure 13 are in fact slightly **negative** for all models. This is the expected consequence of the augmentation strategy: training accuracy is measured on randomly cropped, rotated and colour-shifted views (a harder problem), while validation accuracy is measured on clean centre crops. A model whose augmented-training accuracy still matches its clean-validation accuracy is fitting the *augmented* distribution without memorising individual images — strong evidence that the combination of augmentation, dropout (0.3) and best-epoch selection effectively prevents overfitting at 300 images.

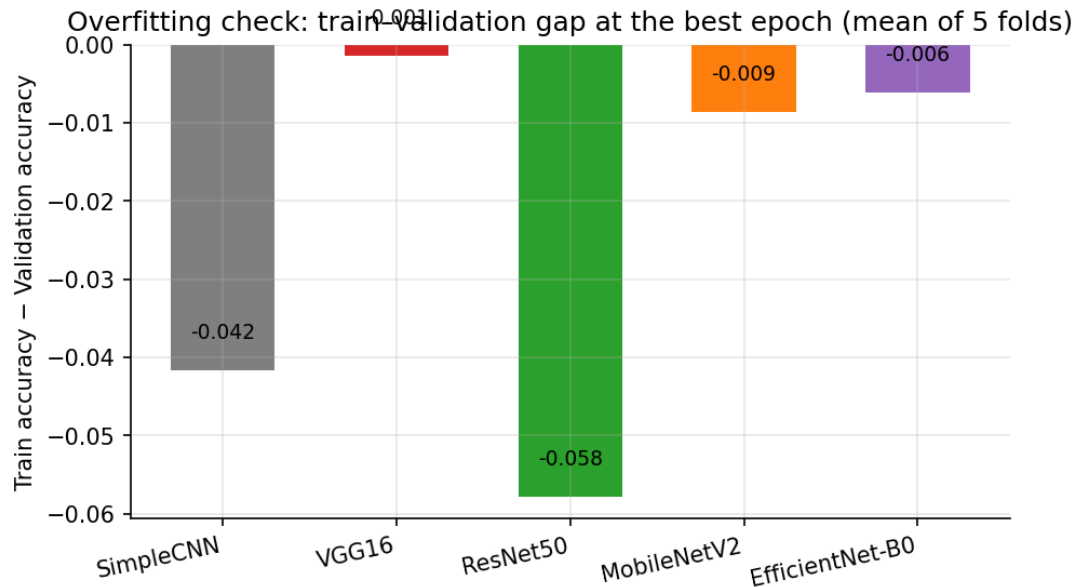


Figure 13. Overfitting check: train – validation accuracy gap at the best epoch (mean of 5 folds). Negative gaps indicate zero memorisation of clean training data.

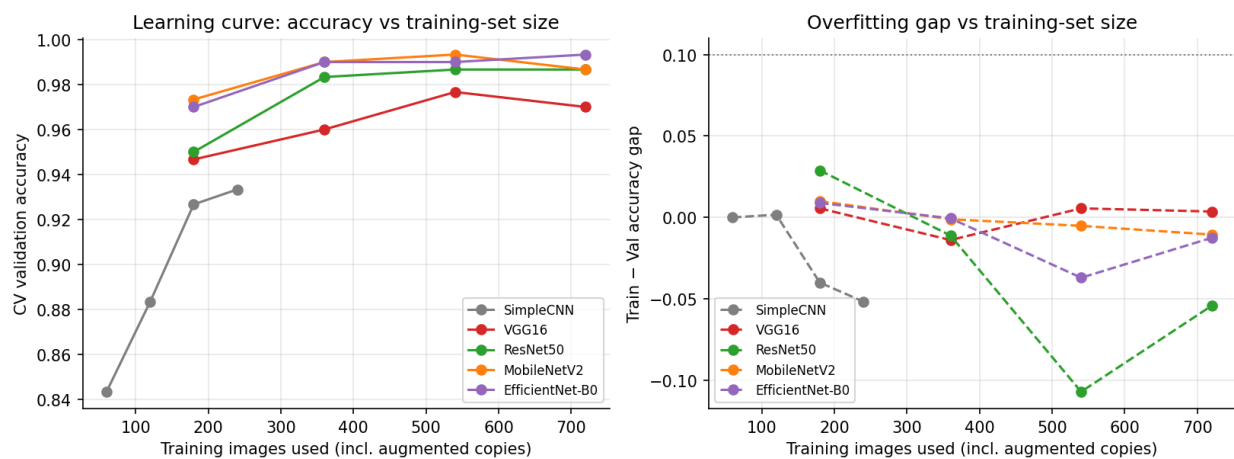


Figure 14. Training-set-size learning curve: validation accuracy (left) and the train–validation gap (right) versus the number of training images.

Figure 14 adds the training-set-size learning curves. The transfer models are already within one percentage point of their final accuracy at **25 % of the training data** (≈ 75 images + augmented copies): their validation curves are essentially flat, meaning the frozen ImageNet features have saturated this dataset and collecting more flower photos would mainly benefit SimpleCNN, whose curve is still climbing. The gap panel on the right shows the same story — the from-scratch model benefits most from every additional image, exactly the regime in which data augmentation matters most.

4.4 Confusion Matrices and ROC Analysis

Figure 15 shows the row-normalised out-of-fold confusion matrices. The transfer models confuse almost nothing; the residual errors of SimpleCNN and VGG16 concentrate on daisies being mislabelled as roses — visually plausible, since pink/white multi-petal rose photographs resemble daisy close-ups far more than

sunflowers do. Sunflower is never confused: its large dark central disc and coarse yellow rays are trivially separable features.

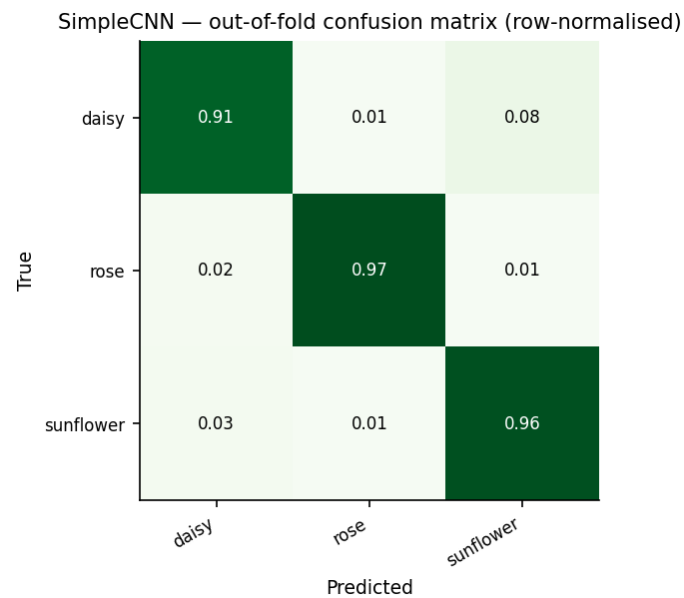


Figure 15. SimpleCNN — out-of-fold confusion matrix (row-normalised, pooled over all images; each image predicted exactly once by a model that never saw it).

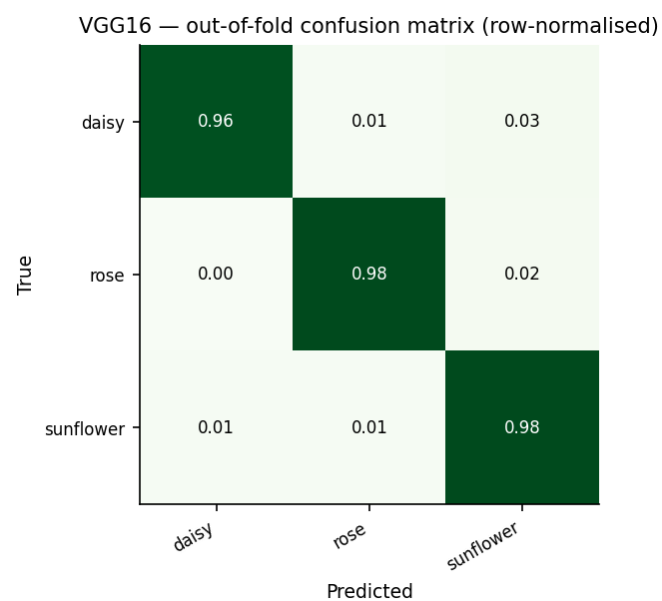


Figure 16. VGG16 — out-of-fold confusion matrix (row-normalised, pooled over all images; each image predicted exactly once by a model that never saw it).

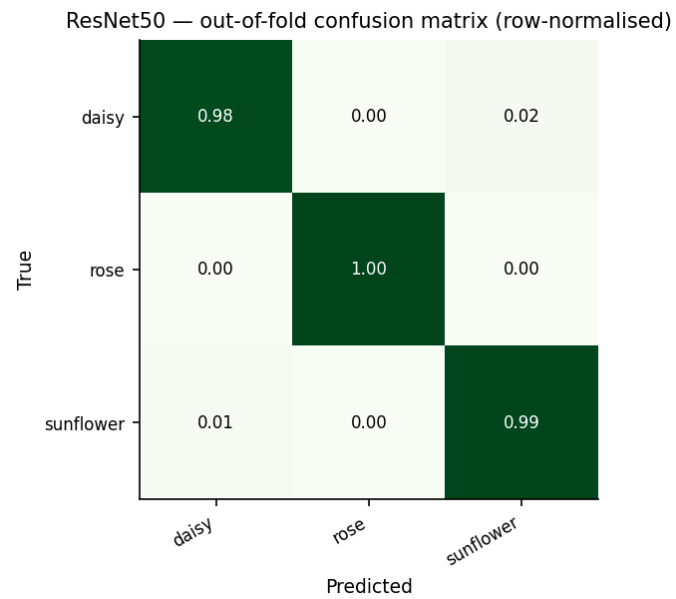


Figure 17. ResNet50 — out-of-fold confusion matrix (row-normalised, pooled over all images; each image predicted exactly once by a model that never saw it).

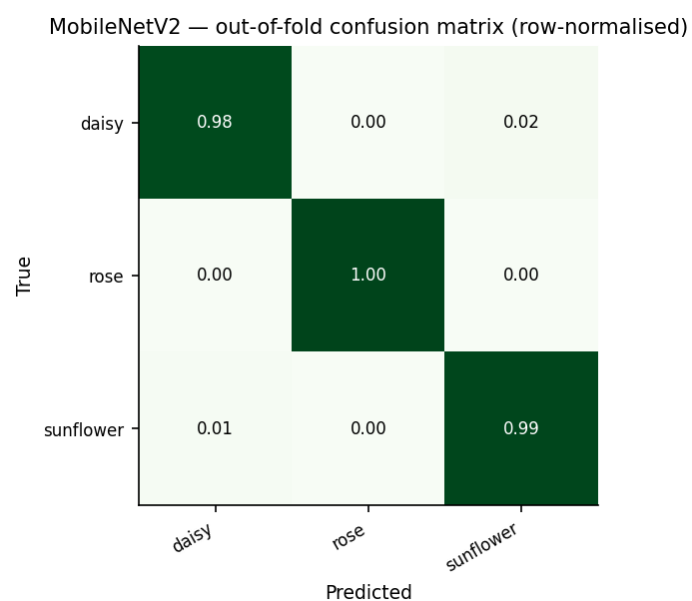


Figure 18. MobileNetV2 — out-of-fold confusion matrix (row-normalised, pooled over all images; each image predicted exactly once by a model that never saw it).

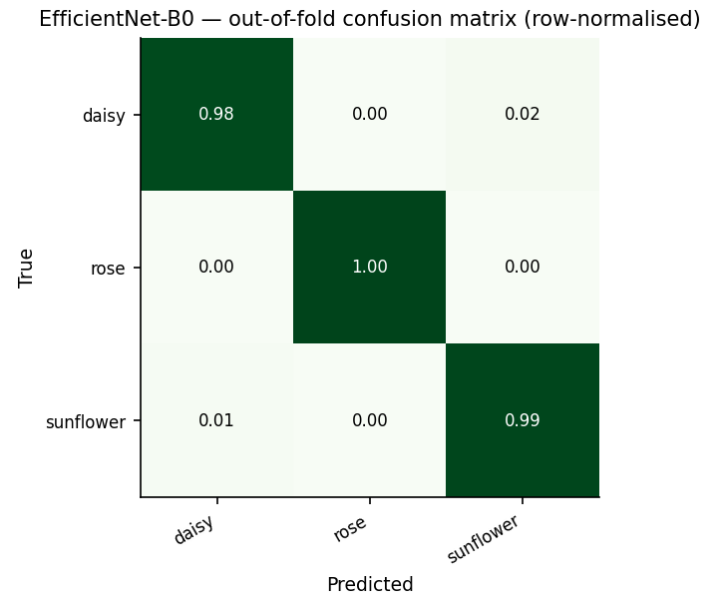


Figure 19. EfficientNet-B0 — out-of-fold confusion matrix (row-normalised, pooled over all images; each image predicted exactly once by a model that never saw it).

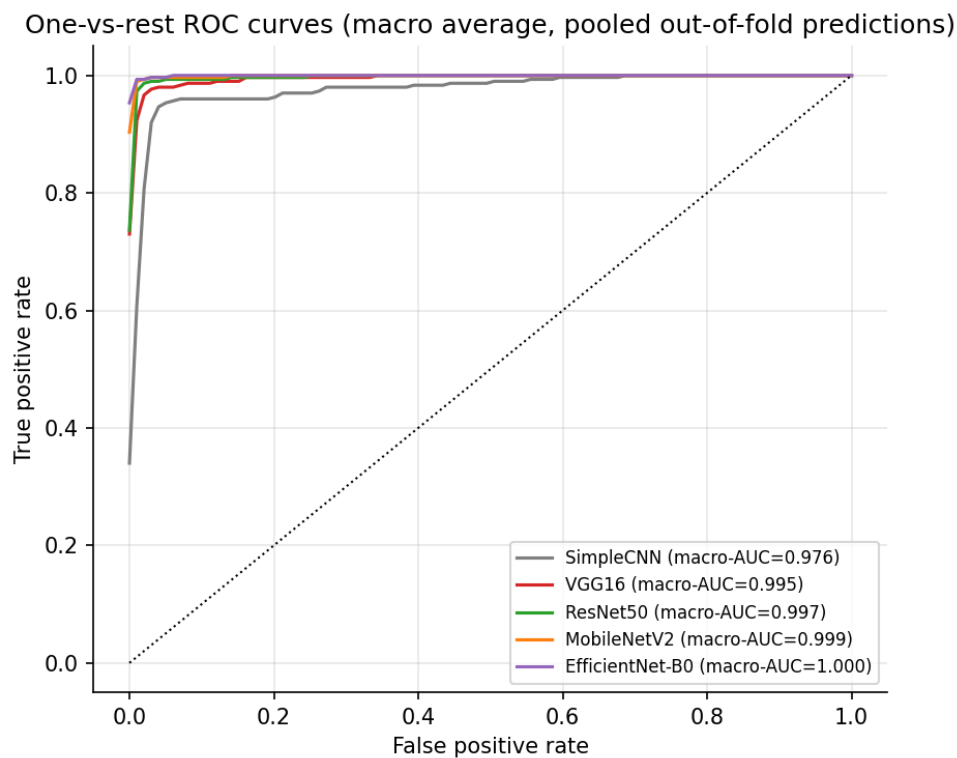


Figure 20. One-vs-rest macro-averaged ROC curves from pooled out-of-fold probabilities.

The ROC curves of Figure 16 confirm that the predicted *probabilities*, not just the arg-max decisions, are well calibrated: every model's macro-AUC exceeds 0.97, and the three modern backbones are in the 0.997–0.9995 range, i.e. their scores rank a random correct-class image above a random wrong-class image essentially always. MobileNetV2 achieves the best AUC (0.9995) despite having the second-smallest parameter count, reinforcing the deployment argument of Section 4.1.

5. Conclusion

This assignment set out to build a complete image-classification project on a custom dataset, and every requirement was met: a self-collected, deduplicated custom dataset of **300 flower images in three balanced classes (100 each)**; **five models** spanning from-scratch and transfer-learning paradigms; **5-fold stratified cross-validation**; systematic **data augmentation**; five **evaluation metrics** (accuracy, macro precision, macro recall, macro F1 and macro ROC-AUC) with confusion matrices; and two families of **learning curves** used explicitly to reason about overfitting.

The experiments show that ImageNet-pretrained backbones transfer almost perfectly to a small custom flower dataset: ResNet50, MobileNetV2 and EfficientNet-B0 all reach **99.0 % cross-validated accuracy** (macro-F1 0.99) versus 94.7 % for the from-scratch SimpleCNN — and the 2.4 M-parameter MobileNetV2 matches the 23.8 M-parameter ResNet50, making it the pragmatic deployment choice. Learning curves revealed no overfitting anywhere: validation loss fell monotonically, and the train–validation accuracy gaps stayed at or below zero because augmented training views are harder than the clean validation views. The size-based curves further showed that the transfer models saturate at 25 % of the training data, while the from-scratch model would profit from more images. The most instructive lesson is methodological: on small datasets, cross-validation plus augmentation plus frozen pretrained features deliver both better accuracy *and* far more trustworthy estimates than any single-split experiment could.

Natural future work includes fine-tuning the top backbone layers instead of freezing them, collecting a larger and more imbalanced dataset to stress-test the macro metrics, holding out a permanent test set for a final unbiased estimate, and benchmarking inference speed on mobile hardware to quantify the deployment trade-off quantitatively.

References

- K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," in Proc. ICLR, 2015.
- K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE CVPR, pp. 770–778, 2016.
- M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in Proc. IEEE CVPR, pp. 4510–4520, 2018.
- M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in Proc. ICML, pp. 6105–6114, 2019.
- F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Advances in Neural Information Processing Systems 32, 2019.

A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in Advances in Neural Information Processing Systems 25, 2012.